



MET CS 688

Graph Mining Algorithms II

Leila Ghaedi – Fall 2024

Creator: cemagraphics | Credit: Getty Images/iStockphoto



A* Algorithm - Finding the shortest path

- It can be called an extended version of Dijkstra with an additional heuristic. A* introduces a score, which is the sum of the cost of the path, and a heuristic (which is the end node in the path) search.
- A* starts by using a piece of prior knowledge about the distances between the target node and other nodes in the graph. This distance that is coming from an estimation is called heuristic distance $h(n)$. To remember the objective of heuristics distances, keep in mind they are guessed distances and not actual distances. It means we should have some prior knowledge to have a good estimation of heuristic distances, such as the approximate weight of edges.
- For example, we are developing a robot navigation app in the city to deliver food. There are some streets that have lots of bumps or people who hack into the robot to steal its food. The cost of passing those streets is higher than other streets and this could be added as heuristic distance.



A* Algorithm

1.Initialization: Similar to Dijkstra, the A* algorithm starts by initializing the open set with the start node, whose cost is zero, and the closed set (to keep track of visited nodes).

2.Heuristic Function ($h(n)$): A* introduces a heuristic estimate ($h(n)$) of the shortest path from a node n to the goal. This heuristic function must be admissible (never overestimating the actual cost) and often takes the form of spatial distance like Euclidean or Manhattan distance.

3.Process Nodes:

1. Similar to Dijkstra, remove the node with the lowest $f(n)$ from the open set, where $f(n) = g(n) + h(n)$, $g(n)$ being the cost to reach node n from the start.
2. For each neighbor of this node, calculate its tentative $g(n)$. If the new $g(n)$ is lower than the recorded value, update it and set its parent to the current node.
3. Add neighbors to the open set if they're not already there or update their positions if $f(n)$ value changes.

4.Goal Check and Path Retrieval: Repeat until the goal node is the node being processed. Reconstruct the path from the recorded parent or previous nodes.

A* Algorithm vs Dijkstra

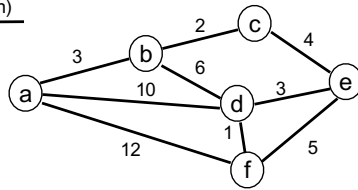
A* is generally more efficient than Dijkstra when a good heuristic is available because it intelligently guides the search in the direction of the goal, potentially exploring fewer nodes. Its computational complexity also depends on the choice of heuristic but often results in faster performance than Dijkstra's in practice when the heuristic effectively estimates the remaining cost. Both algorithms are foundational in fields ranging from robotics to network routing, with A* favored in real-time systems due to its potential speed improvements over Dijkstra's in scenarios where an informed estimate of cost can be made.

A* Algorithm

- The quality of the heuristic influences the algorithm's accuracy as well. A good heuristic distance should be close to actual distance, but getting a high-quality heuristic distance estimate is not usually easy. *If all heuristic distances are set to zero, it works exactly as Dijkstra algorithm.*
- $g(n)$, is an actual distance on the path from a source vertex to the target vertex (for example weights on edges) and $h(n)$ is the guessed distance. The algorithm does not decide just by $g(n)$, it decides based $f(n)$, which is the sum of actual distance and heuristic distance $f(n) = g(n) + h(n)$.

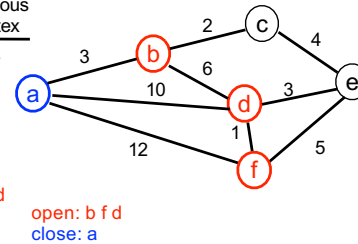
①

node	$h(n)$	$g(n)$	$f(n)$
a	3		
b	4		
c	6		
d	6		
e	12		
f	8		



②

node	$h(n)$	$g(n)$	$f(n)$	previous vertex
a	3	0	3	—
b	4	3	7	a
c	6	6 9	∞ 15	a b
d	6	10 6	∞ 16	a b
e	12	∞ 12	∞ 20	a d
f	8	12 9	∞ 18	a d

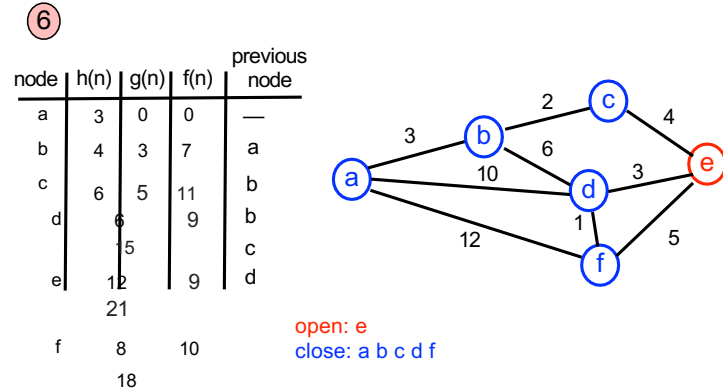
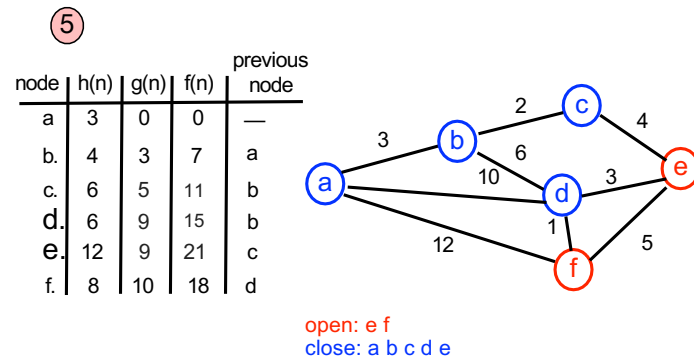
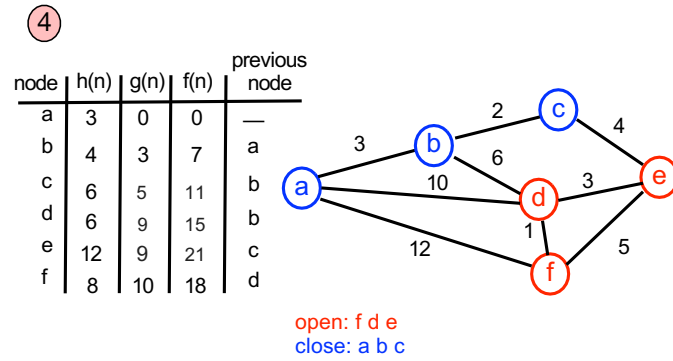
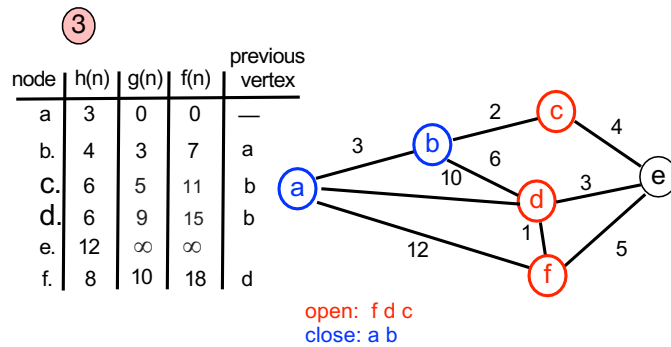


We want to find the distance of 'a' to other nodes.

It adds vertex 'a' into a list of open vertices, then it checks adjacent nodes to vertices to 'a' which are 'b', 'd', and 'f' and calculates their $f(n)$ distance.

It moves 'a' into the list of close vertices and 'b', 'd' and 'f' vertices into the list of open vertices, as it is shown in Figure 2. Now, the algorithm should select a new current vertex from the list of open vertices. It selects vertex 'b', because from all vertices inside open list b's $f(n)$ distance is the lowest one, equal to 7. Then, it calculates $f(n)$ for all adjacent vertices of 'b' (vertices 'd' and 'c', but not 'a' because it is in the close list) and adds them all to the list of open vertices. Figure 2 presents an additional column (previous vertex), which specifies from which vertex the algorithm reaches that particular vertex.

Besides, while processing 'b' the algorithm realizes that distance from $a \rightarrow b \rightarrow d$ is shorter than $a \rightarrow d$ and thus there is a shorter path existing and updates the row of data from node d. The same story is applicable for vertex 'f' as well. The path from 'a' to 'f' via 'd' is shorter and thus the path from $a \rightarrow f$ will be substituted via $a \rightarrow d \rightarrow f$. We mark these changes in Figure 2 in red on the table.



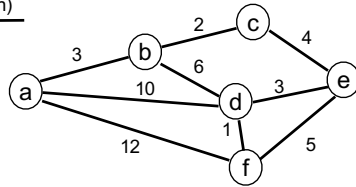
The algorithm moves 'b' into the close list and chooses the vertex that has the smallest $f(n)$ as the next vertex, i.e. 'c'. Then, it calculates the distances of adjacent vertices to 'c' (distances from 'a' to adjacent 'c' vertices), as is shown in Figure 3.

Next, the shortest 'c' is moved to the close list, and distances from 'c' adjacent vertices that are not in the close list, which is 'd' will be selected (see Figure 4).

After node 'd' has been selected for processing, all distances from 'a' to its adjacent nodes, including 'f' and 'e' will be calculated. Since 'f' is the shortest one, the algorithm selects 'f' (see Figure) and then it selects 'e' (see Figure 6). Nothing will change, all nodes are processed and by looking at column $f(n)$ the algorithm outputs the distances to all nodes.

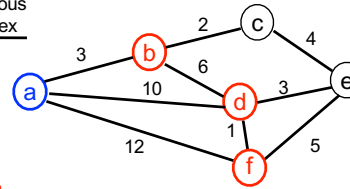
1

node	h(n)	g(n)	f(n)
a	3		
b	4		
c	6		
d	6		
e	12		
f	8		



2

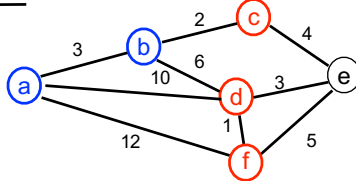
node	h(n)	g(n)	f(n)	previous vertex
a	3	0	3	—
b	4	3	7	a
c	6	6+3	∞	b
d	6	10	16	a
e	12	∞	∞	a
f	8	12	20	a



open: b f d
close: a

3

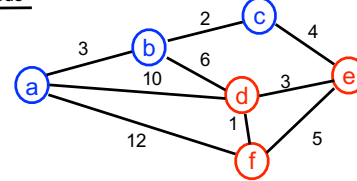
node	h(n)	g(n)	f(n)	previous vertex
a	3	0	0	—
b	4	3	7	a
c	6	5	11	b
d	6	9	15	a
e	12	∞	∞	
f	8	10	18	d



open: f d c
close: a b

4

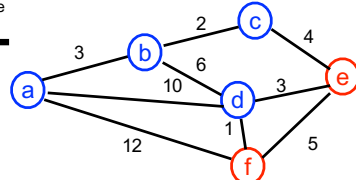
node	h(n)	g(n)	f(n)	previous node
a	3	0	0	—
b	4	3	7	a
c	6	5	11	b
d	6	9	15	b
e	12	9	21	c
f	8	10	18	d



open: f d e
close: a b c

5

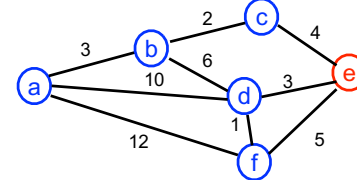
node	h(n)	g(n)	f(n)	previous node
a	3	0	0	—
b	4	3	7	a
c	6	5	11	b
d	6	9	15	b
e	12	9	21	c
f	8	10	18	d



open: e f
close: a b c d e

6

node	h(n)	g(n)	f(n)	previous node
a	3	0	0	—
b	4	3	7	a
c	6	5	11	b
d	6	9	15	b
e	12	9	21	c
f	8	10	18	d

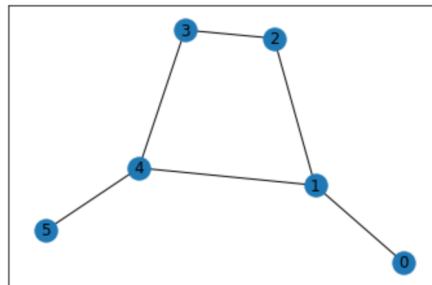


open: e
close: a b c d f

A* Algorithm Python Example

```
In [4]: G2=nx.Graph()
# form a weighted graph
elist=[(0,1,10), (1,2,10),
        (2,3,10), (3,4,10),
        (4,5,10), (1,4,3) ]
G2.add_weighted_edges_from(elist)
nx.draw_networkx(G2)
# Find the shortest path from 0 to 5 using A* search
path = nx.astar_path(G2, source=0, target=5)
# Print the path
print(path)
```

```
[0, 1, 4, 5]
```



```
In [5]: # without applying the heuristic dijkstra and A* are the same
print(nx.dijkstra_path(G2, 0, 5))
```

```
[0, 1, 4, 5]
```

A* Algorithm Python Example

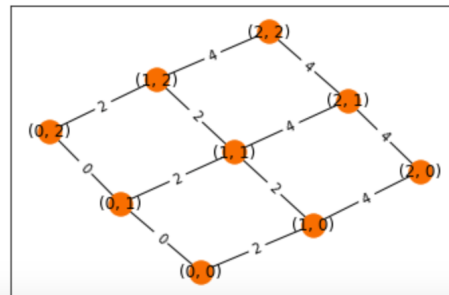
```
In [6]: # this function is used to assign heuristic distance h(n)  
def dist(a,b):  
    (x1,y1) = a  
    (x2,y2) = b  
    return (((x1-x2) ** 2 + (y1-y2) ** 2) ** 0.5)
```

A* Algorithm Python Example

```
In [7]: G3 = nx.grid_graph(dim=[3, 3])
# nodes are two-tuples (x,y)
nx.set_edge_attributes(G3, {e: e[1][0]*2 for e in G3.edges()}, "cost")
path = nx.astar_path(G3, (0,0), (2,2), heuristic=dist, weight="cost")
length = nx.astar_path_length(G3, (0,0), (2,2), heuristic=dist, weight="cost")
print('Path:')
print(path)
print("Path length: ", length)

pos=nx.spring_layout(G3)
nx.draw_networkx(G3, pos, with_labels = True , node_color="#f86e00")
edge_labels=nx.get_edge_attributes(G3, "cost")
nx.draw_networkx_edge_labels(G3, pos, edge_labels=edge_labels)
plt.show()

Path:
[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2)]
Path length: 6
```



Outline

- Basic Graph Concepts
- Minimum Spanning Tree
- Shortest Path Finding
- **Centrality Measurements (Degree, Closeness, Betweenness, Eigenvector)**
- Link Analysis
- Community Detection

Centrality Measurements

- A network is composed of nodes and nodes have different impacts on the network, some nodes are more important, and some are less important inside a network.
- Centrality measures fall into two main categories: **radial and medial measures**.
 - Radial measures encompass degree, closeness, and eigenvector centrality.
 - Medial measures encompass betweenness.

Centrality Measurements

- Degree Centrality
- Closeness Centrality
- Betweenness Centrality
- Eigenvector Centrality

Degree Centrality

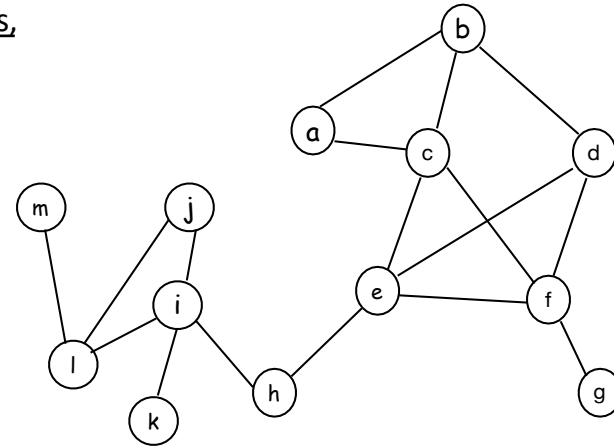
- The simplest form of centrality is degree (node) centrality, which finds popular nodes in a graph by simply counting the number of incoming and outgoing edges (links) to all nodes, then the algorithm selects the node with the highest number of degrees as a central node of the graph. The equation to calculate degree (CD_i) centrality is written as follows:

- $$CD_i = \frac{\sum_{j=1}^n a_{ij}}{n - 1}$$

- In this equation, n presents the number of nodes and a_{ij} is the number of connections from node i to other nodes.

- The degree centrality of node c is calculated as:

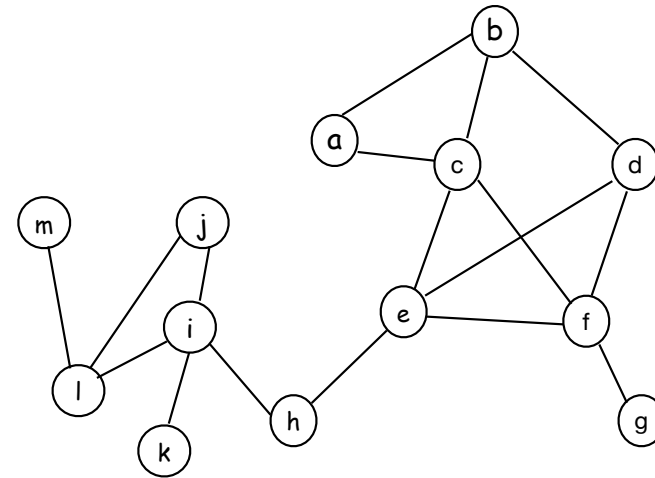
$$CD_i = \frac{4}{12}$$



Closeness Centrality

- It is used to identify nodes that distribute information through the network efficiently. Closeness centrality measures the average distance from a node to all other nodes in the network. The low closeness centrality means the node is located in the center of a graph and can access all other nodes with fewer hops.

- $$CC_i = \frac{n-1}{\sum_{j=1}^n d(i, j)}$$
- In the above equation, n is the number of nodes in the graph and $d(i, j)$ is the shortest path from the source node i to target node j .
- Let's say we would like to calculate the closeness centrality of node 'f'. First, we list shortest distances of f to all other nodes as follows: {a:2, b:2, c:1, d:1, e:1, g:1, h:2, i:3, j:4, k:4, l:4, m:5}.
Therefore, the result will be $(13 - 1)/30 = 0.4$



Betweenness Centrality

- It assigns a score to a node, based on the locations of a node on the shortest path between other nodes of the graph. In simple words, it is used to identify nodes that act as a bridge to connect other nodes together.
- Nodes with high betweenness centrality are having a higher influence on the graph to connect communities or clusters of nodes together. These nodes (with high betweenness centrality) have higher influences in distributing the information in the graph because they are responsible for passing information in the graph.

Betweenness Centrality

The equation to calculate betweenness centrality is written as:

$$BD_k = \sum_{i \neq j \neq k} \frac{d_{i,j}(k)}{d_{i,j}}$$

In this equation $d_{i,j}$ is the shortest path from node i to j (geodesic path from i to j), and $d_{i,j}(k)$ is the number of those paths that pass through k . Note that node k as start or end node does not count, it should be between i and j . In simple words, we can write betweenness centrality as:

$$BD_k = \sum \frac{\text{number of shortest paths that pass } k}{\text{all shortest paths between every node}}$$

Eigenvector Centrality

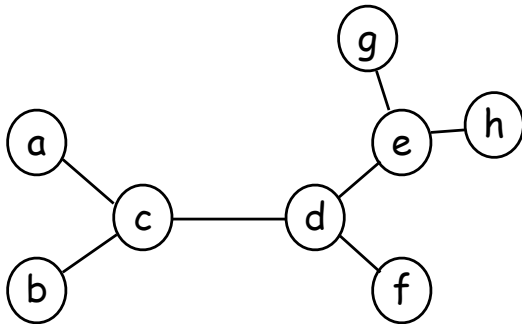
- It measures/quantifies the influence of a node based on the connected nodes to this node.
- Assume we have node X and Y both have the same number of connections, Eigen-vector centrality score prioritize X over Y , if X connected to nodes with a high number of connections, rather than Y that is connected to the same number of nodes, but those nodes have a lower number of connections.
- The concept of eigen-vector centrality leads to the introduction of web search revolutionary algorithms, Page rank and HITS.
- This algorithm uses a matrix that describes the node relation and calculate the eigen vector of this matrix.

Eigenvector Centrality

- Let's say we have a small network with four individuals: A, B, C, and D. The connections are as follows:
- A is friends with B and C.
- B is friends with A and C.
- C is friends with A and B.
- D is not connected to anyone.
- The eigenvector centrality is found by solving the equation $Ax = \lambda x$, where A is the adjacency matrix, x is the eigenvector, and λ is the eigenvalue.
- A: 0.5, B: 0.5, C: 0.5, D:0.0

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	0
D	0	0	0	0

Centrality Examples

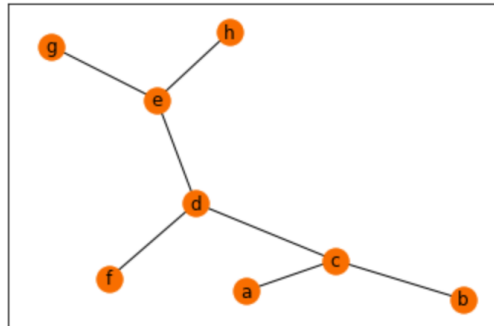


Node	degree	closeness	betweenness	Eigen Vector
a	0.14	0.37	0.00	0.22
b	0.14	0.37	0.00	0.22
c	0.43	0.54	0.52	0.46
d	0.43	0.64	0.71	0.56
e	0.43	0.54	0.52	0.46
f	0.14	0.41	0.00	0.26
g	0.14	0.37	0.00	0.22
h	0.14	0.37	0.00	0.22

Centrality Example

```
In [1]: import networkx as nx  
import matplotlib.pyplot as plt
```

```
In [2]: G=nx.Graph()  
# form graph  
elist=[('a','c'), ('c','b'),  
        ('c','d'), ('d','e'),  
        ('d','f'), ('e','g'),  
        ('e','h')] ]  
G.add_edges_from(elist)  
nx.draw_networkx(G,node_color="#f86e00")
```



Centrality Example

```
In [3]: d_centrality = nx.degree centrality(G)
print(['%s %0.2f'%(node,d_centrality[node]) for node in d_centrality])

['a 0.14', 'c 0.43', 'b 0.14', 'd 0.43', 'e 0.43', 'f 0.14', 'g 0.14', 'h 0.14']
```

```
In [4]: c_centrality = nx.closeness centrality(G)
print(['%s %0.2f'%(node,c_centrality[node]) for node in c_centrality])

['a 0.37', 'c 0.54', 'b 0.37', 'd 0.64', 'e 0.54', 'f 0.41', 'g 0.37', 'h 0.37']
```

```
In [5]: b_centrality = nx.betweenness centrality(G)
print(['%s %0.2f'%(node,b_centrality[node]) for node in b_centrality])

['a 0.00', 'c 0.52', 'b 0.00', 'd 0.71', 'e 0.52', 'f 0.00', 'g 0.00', 'h 0.00']
```

```
In [6]: e_centrality = nx.eigenvector centrality(G)
print(['%s %0.2f'%(node,e_centrality[node]) for node in e_centrality])

['a 0.22', 'c 0.46', 'b 0.22', 'd 0.56', 'e 0.46', 'f 0.26', 'g 0.22', 'h 0.22']
```

Outline

- Basic Graph Concepts
- Minimum Spanning Tree (Prim, Kruskal)
- Shortest Path Finding (Dijkstra)
- Centrality Measurements (Degree, Closeness, Betweenness, EigenVector)
- **Link Analysis** (Page Rank, HITS)
- Community Detection (Louvain, Leiden)

Page Rank

Page Rank

- Page Rank Algorithm is named after both the term "web page" and Google's co-founder Larry Page.
- One of the most important and recognized applications of the computer science is web search, which leads to the introduction of search engines.
- Google's search engine is not the first search engine introduced in the market, but it is the most successful one that has defeated spammers.
- The first version of Google search engine was **Page Rank**. Nevertheless, spammers find a way to spam it as well. Then google introduced **Trust Rank**.

Spammers and Search Engines

... if you were selling shirts on the Web, all you cared about was that people would see your page, regardless of what they were looking for. Thus, you could add a term like “movie” to your page, and do it thousands of times, so a search engine would think you were a terribly important page about movies. When a user issued a search query with the term “movie,” the search engine would list your page first. To prevent the thousands of occurrences of “movie” from appearing on your page, you could give it the same color as the background. And if simply adding “movie” to your page didn’t do the trick, then you could go to the search engine, give it the query “movie,” and see what page did come back as the first choice. Then, copy that page into your own, again using the background color to make it invisible.

Source: Leskovec, J., Rajaraman, A., & Ullman, J. D. (2014). *Mining of massive datasets*. Cambridge university press.

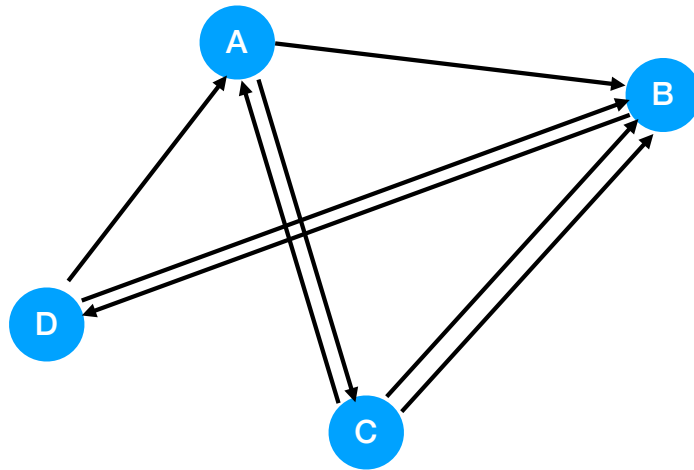
How the first version of Page Rank resolves spam problem

1. Pages that have a large number of visits, considered to be more important and appear earlier in the search, on top of the pages that have a smaller number of visits.
2. The terms (keywords) inside the page are not used to judge the content of the page alone. In addition, it uses the terms used in the links to that particular page as well. what you say about yourself is not enough, what others say about you is also important as well.
3. The fundamental idea behind Page Rank is based on the concept of "link analysis".
4. The basic idea is that a webpage is considered more important if it is linked to by other important webpages. The algorithm views a link from one page to another as a vote of confidence or endorsement. However, not all votes are equal; the importance of the page casting the vote also matters.

Page Rank

- **Link as a Vote:**
 - If Page A has a link to Page B, it is considered as Page A "voting" for Page B.
 - The more inbound links a page receives, the more "votes" it accumulates.
- **Quality of Votes:**
 - Not all votes are equal. The importance of the page casting the vote affects the weight of the vote.
 - A link from a highly ranked or influential page contributes more to the receiving page's PageRank.
- **Damping Factor:**
 - The algorithm introduces a damping factor, typically set to 0.85, to model the probability that a user randomly clicking on links will continue to browse.
- **Recursive Calculation:**
 - PageRank is calculated iteratively. Each iteration represents a hypothetical "user" moving through the web by following links.
 - The algorithm assigns an initial PageRank value to each page, and these values are updated in each iteration based on the link structure of the web.

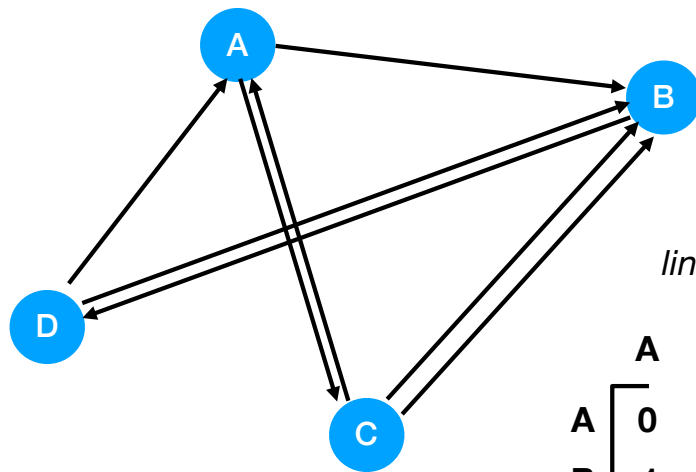
Page Rank



- The objective of page rank is to determine the web page importance for the search engine.



Simplified Page Rank



link matrix

	<i>from</i>			
	A	B	C	D
A	0	0	1	1
B	1	0	2	1
C	1	0	0	0
D	0	1	0	0

to

Out-degree

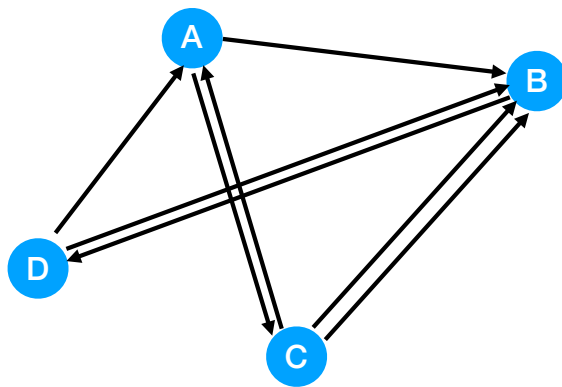
A = 2

B = 1

C = 3

D = 2

Simplified Page Rank



link count

	A	B	C	D
A	0	0	1	1
B	1	0	2	1
C	1	0	0	0
D	0	1	0	0

Out-degree

A = 2

B = 1

C = 3

D = 2

Transition Matrix

	A	B	C	D
A	0	0	1/3	1/2
B	1/2	0	2/3	1/2
C	1/2	0	0	0
D	0	1/1	0	0

Transition Matrix node:
(link count) ÷ (out degree of columns)

Page Rank

- The page rank algorithm promotes a web page is as follows:
- The importance of the web pages pointing to a web page is more important than the number of links points to that page.
- In other words, one important page is more important than several other low ranked pages pointing to a website.

Page Rank Equation

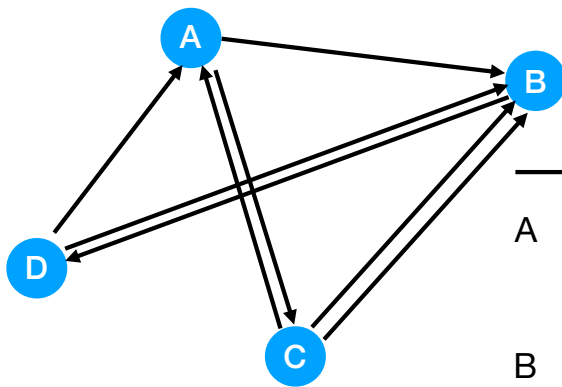
$$PR_{t+1}(P_i) = \sum_{P_j} \frac{PR_t(P_j)}{C(P_j)}$$

Page Rank of the $t+1$ iteration

Page Rank of the t iteration (previous)

Number of out links

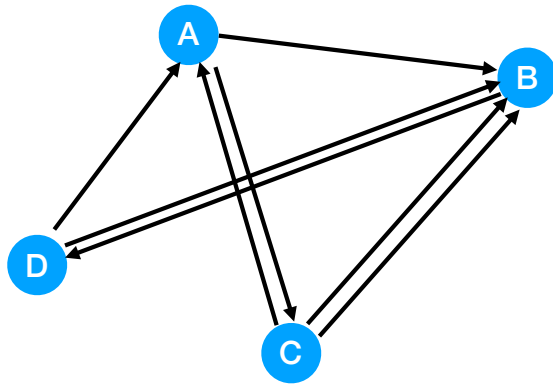
Page Rank



	Iteration 0	Iteration 1
A	1/4	$\frac{\text{PR}(C) + \text{PR}(D)}{\text{out}(C) + \text{out}(D)} = \frac{1/4}{3} + \frac{1/4}{2} = \frac{20}{96} = 0.208$
B	1/4	$2 \times \frac{\text{PR}(C) + \text{PR}(D)}{\text{out}(C) + \text{out}(D)} + \frac{\text{PR}(A)}{\text{out}(A)} = 2 \times \frac{1/4}{3} + \frac{1/4}{2} + \frac{1/4}{2} = \frac{40}{96} = 0.416$
C	1/4	$\frac{\text{PR}(A)}{\text{out}(A)} = \frac{1/4}{2} = \frac{1}{8} = \frac{12}{96} = 0.125$
D	1/4	$\frac{\text{PR}(B)}{\text{out}(B)} = \frac{1/4}{1} = \frac{1}{4} = \frac{24}{96} = 0.25$

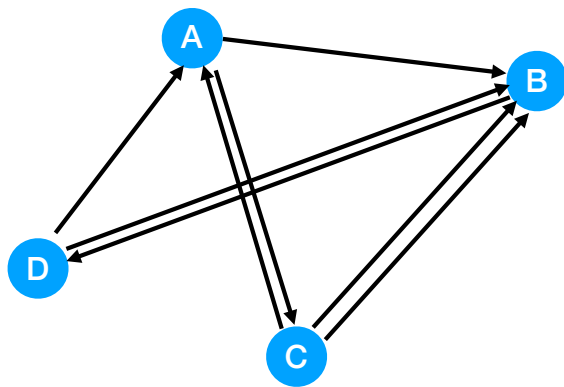
initial page rank should be all equal to one. Also once the PageRank calculations have settled down, the “normalized probability distribution” (the average PageRank for all pages) will be 1.0.

Page Rank



	Iteration 0	Iteration 1	Iteration 2
A	1/4	$\frac{20}{96}$	$\frac{\frac{12}{96}}{3} + \frac{\frac{24}{96}}{2} \sim 0.166$
B	1/4	$\frac{40}{96}$	$2 \times \frac{\frac{12}{96}}{3} + \frac{\frac{24}{96}}{2} + \frac{\frac{20}{96}}{2} \sim 0.312$
C	1/4	$\frac{12}{96}$	$\frac{\frac{20}{96}}{2} \sim 0.104$
D	1/4	$\frac{24}{96}$	$\frac{\frac{40}{96}}{1} \sim 0.416$

Page Rank



	It. 0	It. 1	It. 2	Page Rank
A	1/4	$\frac{20}{96}$	~0.166	3
B	1/4	$\frac{40}{96}$	~0.312	2
C	1/4	$\frac{12}{96}$	~0.104	4
D	1/4	$\frac{24}{96}$	~0.416	1

<https://www.youtube.com/watch?v=IKXvSKal2Ko>

Page Rank

- When a web page does not have an outbound link, we can not use it in Page Rank calculation. These pages called **dangling link** and they should be removed before the page rank calculation.
- We can not round page rank values, we must always use absolute values, otherwise we get into a problem.
- **Teleport** or **damping factor** is used that when a page has no inbound link, it receives some rank score, e.g. $(1-d)=0.15$ if $d=0.85$. Therefore, the lowest page rank will be 0.15.
- Each iteration provides a more accurate result than previous iteration.

Page Rank (Original Formula)

PR(A): Page rank of web page A

d: Damping Factor

PR(T_x): Page Rank of page T_x, which link to page A.

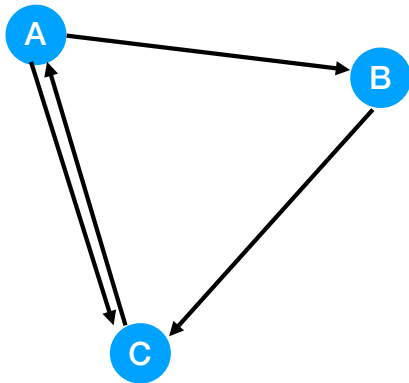
C(T_x): Number of outbound link to Page A.

- *Page A has pages T₁...T_n which point to it (i.e., citations). The parameter d is a damping factor which can be set between 0 and 1. We usually set d to 0.85. Also $C(A)$ is defined as the number of links going out of page A. The PageRank of a page A is given as follows:
$$PR(A) = (1-d) + d (PR(T_1)/C(T_1) + \dots + PR(T_n)/C(T_n))$$*

Page Rank (Original Formula)

- *Page A has pages $T_1 \dots T_n$ which point to it (i.e., citations). The parameter d is a damping factor which can be set between 0 and 1. We usually set d to 0.85. Also $C(A)$ is defined as the number of links going out of page A. The PageRank of a page A is given as follows:
$$PR(A) = (1-d) + d (PR(T_1)/C(T_1) + \dots + PR(T_n)/C(T_n))$$*
- *Note that the PageRank forms a probability distribution over web pages, so the sum of all web pages in PageRanks will be one or very close to one.*
- *PageRank or $PR(A)$ can be calculated using a simple iterative algorithm, and corresponds to the principal eigenvector of the normalized link matrix of the web.*
- **Damping factor** is used that even if the page has no inbound link, it receives some rank score, e.g. $(1-d)=0.15$ if $d=0.85$. Therefore, the lowest page rank will be 0.15.

Page Rank



Damping Factor=0.85

$$PR(X) = (1-d) + d (PR(T_1)/C(T_1) + \dots + PR(T_n)/C(T_n))$$

- If the PR is not provided, the initial PR for all pages is equal to 1.

Iteration 1:

initial PR

$$PR(A) = (1-0.85) + 0.85 (1/1) = 1$$

$$PR(B) = (1-0.85) + 0.85 (1/2) = 0.575$$

$$PR(C) = (1-0.85) + 0.85 ((1/2)+(0.575/1)) = 1.06375$$

Iteration 2:

$$PR(A) = (1-0.85) + 0.85 (1.06375/1) = 1.0541875$$

$$PR(B) = (1-0.85) + 0.85 (1.0541875/2) = 0.598029875$$

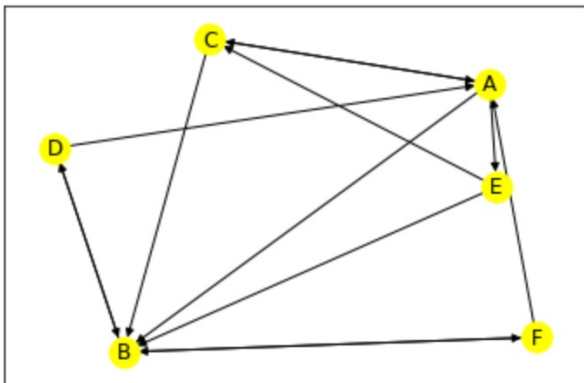
$$PR(C) = (1-0.85) + 0.85 ((1.0541875/2)+(0.598029875/1)) = 1.06354922$$

Page Rank Example

```
In [1]: import networkx as nx  
import matplotlib.pyplot as plt
```

```
In [3]: G=nx.DiGraph()  
G.add_edges_from([('A','B'),('A','C'),('D','A'),('D','B'),('B','D'),('C','A'),('C','B'),('C','B'),  
                  ('F','B'),('E','B'),('C','B'),('F','A'),('B','F'),('A','E'),('E','C'),('F','A')])  
nx.draw_networkx(G,with_labels=True,node_color='yellow')  
pr=nx.pagerank(G,alpha=0.4)  
pr
```

```
Out[3]: {'A': 0.18921372753470506,  
        'B': 0.2394879728899863,  
        'C': 0.1502742453552812,  
        'D': 0.14789772740301782,  
        'F': 0.14789772740301782,  
        'E': 0.12522859941399175}
```



HITS (Hyperlink-Induced Topic Search)

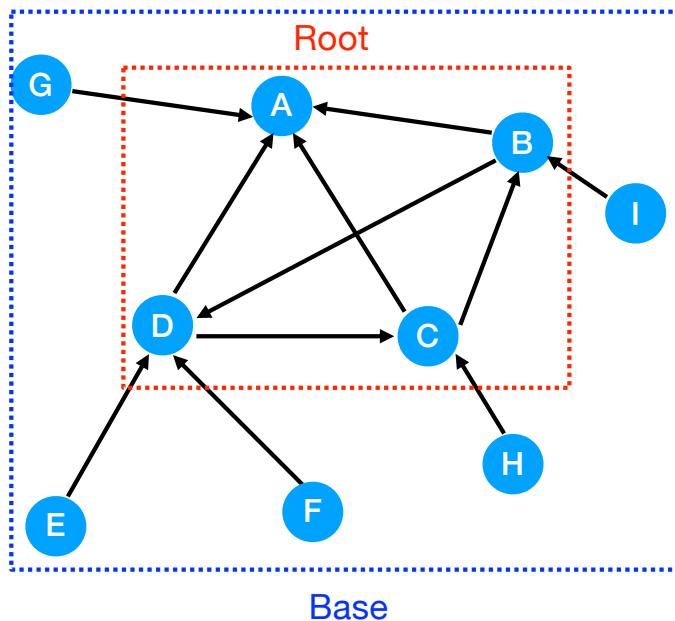
HITS Algorithm

- This algorithm is also designed to deal with page ranking and web searches.
- Page rank uses one factor (links to that page) to calculate the importance of a page. HITS has utilized two factors, Hubs and Authorities:
 - 1. Authorities** are webpages that are linked to by many hubs. They are considered as pages with valuable content on a particular topic.
 - 2. Hubs** are webpages that link to many other pages. They are considered as good resources for finding information but may not contain the specific content a user is seeking.
- HITS works based on this principles: *“a page is a good hub if it links to a good authorities, and a page is a good authority if it is linked by good hubs”*.

HITS Algorithm

- There are two scores assigned to a page:
 - Hub Score
 - Authority
- Both values are between 0 and 1.
- Another difference it has with page rank is that instead of using the entire network it uses a subset of the network (root set).

HITS Algorithm

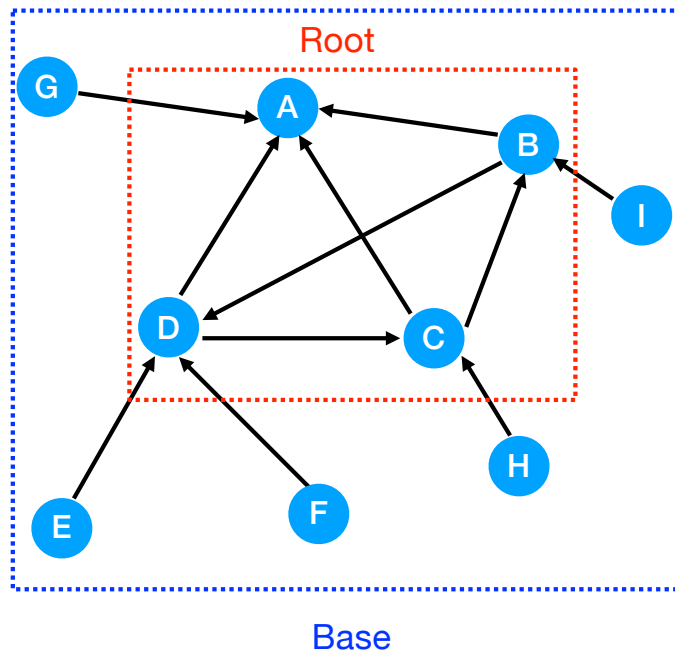


- HITS works based on this principle: “a page is a good hub if it links to good authorities, and a page is a good authority if it is linked to good hubs”.
- A set of highly relevant pages (pages that receive lots of queries) are potential authorities and they have stayed in a **root** set.
- Nodes that are pointing to root set nodes stayed in a set called **base** set.

HITS Algorithm

- Similar to page rank it operates iteratively.
 1. The algorithm assigns authority score of 1 to each node.
 2. It calculates the authority score by counting the number of node links to each authority node.
 3. It calculates the hub score by counting the number of node links from authority nodes to each hub nodes.
 4. It normalizes both authority and hub scores.
 5. It repeats the process iteratively (the number of iteration depends on our choice).

HITS Algorithm

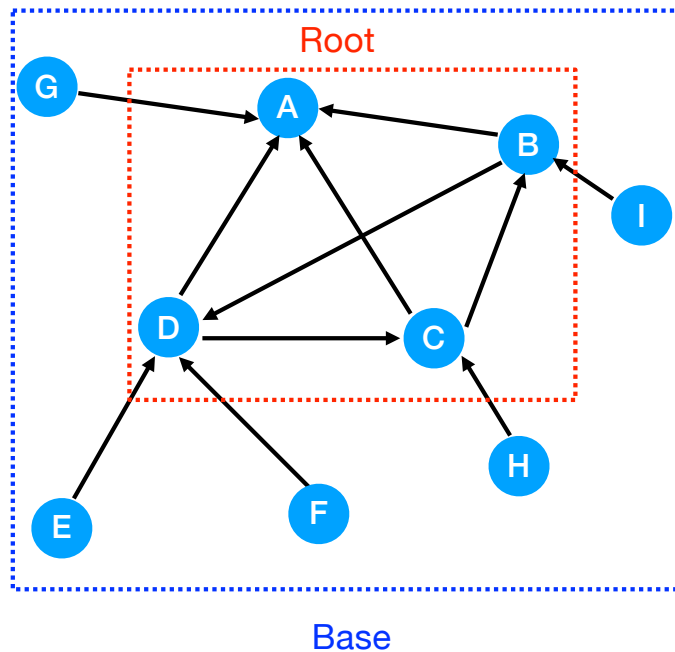


	Iteration 0		Iteration 1	
	Auth.	Hub.	Auth.	Hub.
			Sum of Hubs	Sum of Auths
A	1	1	1+1+1+1	0
B	1	1	1+1	1+1
C	1	1	1+1	1+1
D	1	1	1+1+1	1+1
E	1	1	0	1
F	1	1	0	1
G	1	1	0	1
H	1	1	0	1
I	1	1	0	1

HITS Algorithm

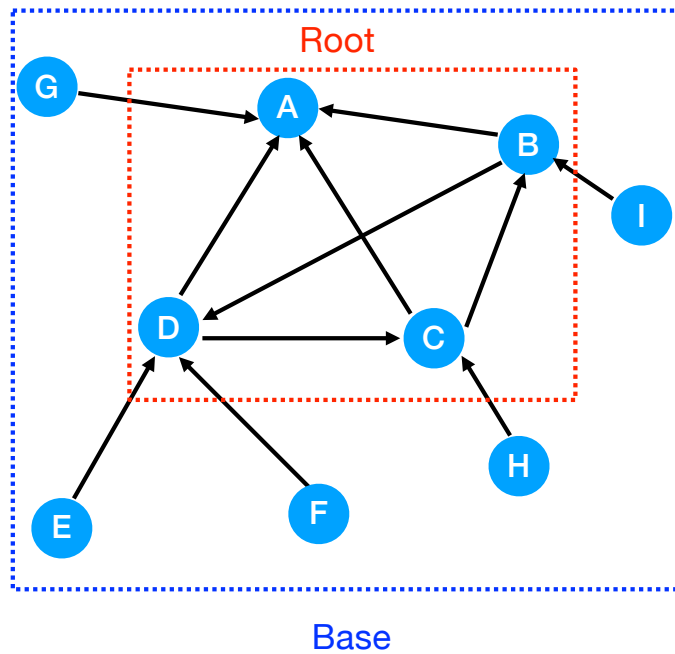
It. 0					It. 1					It. 1					It. 2				
	Auth	Hub	Auth Sum of Hubs	Hub Sum of Auths		Auth	Hub	Auth	Hub		Auth	Hub	Auth	Hub					
A	1	1	1+1+1+1	0	→ Normalize	A	4/11	0											
B	1	1	1+1	1+1		B	2/11	2/11											
C	1	1	1+1	1+1		C	2/11	2/11											
D	1	1	1+1+1	1+1		D	3/11	2/11											
E	1	1	0	1		E	0	1/11											
F	1	1	0	1		F	0	1/11											
G	1	1	0	1		G	0	1/11											
H	1	1	0	1		H	0	1/11											
I	1	1	0	1		I	0	1/11											
sum: 11 sum: 11																			

HITS Algorithm



	It. 1		It. 2	
	Auth	Hub	Auth	Hub
A	4/11	0	$\frac{2}{11} + \frac{2}{11} + \frac{2}{11} + \frac{1}{11}$	
B	$\frac{2}{11}$	$\frac{2}{11}$	$\frac{2}{11} + \frac{1}{11}$	
C	$\frac{2}{11}$	$\frac{2}{11}$	$\frac{2}{11} + \frac{1}{11}$	
D	$\frac{3}{11}$	$\frac{2}{11}$	$\frac{1}{11} + \frac{1}{11} + \frac{2}{11}$	
E	0	$\frac{1}{11}$	0	
F	0	$\frac{1}{11}$	0	
G	0	$\frac{1}{11}$	0	
H	0	$\frac{1}{11}$	0	
I	0	$\frac{1}{11}$	0	

HITS Algorithm

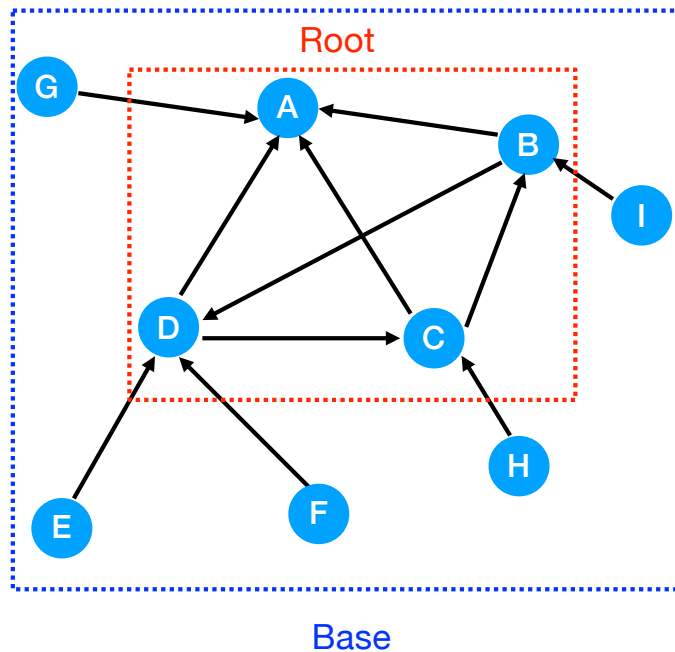


It. 1

It. 2

	Auth	Hub	Auth	Hub
A	4/11	0	$\frac{1}{11} + \frac{2}{11} + \frac{2}{11} + \frac{1}{11}$	0
B	2/11	1/11	$\frac{2}{11} + \frac{2}{11}$	$\frac{4}{11} + \frac{3}{11}$
C	2/11	2/11	$\frac{2}{11} + \frac{1}{11}$	$\frac{4}{11} + \frac{2}{11}$
D	3/11	2/11	$\frac{1}{11} + \frac{1}{11} + \frac{2}{11}$	$\frac{4}{11} + \frac{2}{11}$
E	0	1/11	0	3/11
F	0	1/11	0	3/11
G	0	1/11	0	4/11
H	0	1/11	0	2/11
I	0	1/11	0	2/11

HITS Algorithm



	Iteration 1		Iteration 2	
	Auth	Hub	Auth	Hub
A	4/11	0	$\frac{1}{11} + \frac{2}{11} + \frac{2}{11} + \frac{1}{11}$	0
B	2/11	1/11	$\frac{2}{11} + \frac{2}{11}$	$\frac{4}{11} + \frac{3}{11}$
C	2/11	2/11	$\frac{2}{11} + \frac{1}{11}$	$\frac{4}{11} + \frac{2}{11}$
D	3/11	2/11	$\frac{1}{11} + \frac{1}{11} + \frac{2}{11}$	$\frac{4}{11} + \frac{2}{11}$
E	0	1/11	0	3/11
F	0	1/11	0	3/11
G	0	1/11	0	4/11
H	0	1/11	0	2/11
I	0	1/11	0	2/11

HITS Algorithm

It. 2

	Auth	Hub
A	6/11	0
B	3/11	7/11
C	3/11	6/11
D	3/11	6/11
E	0	3/11
F	0	3/11
G	0	4/11
H	0	2/11
I	0	2/11

sum: 15/11

sum: 33/11

→
Normalize

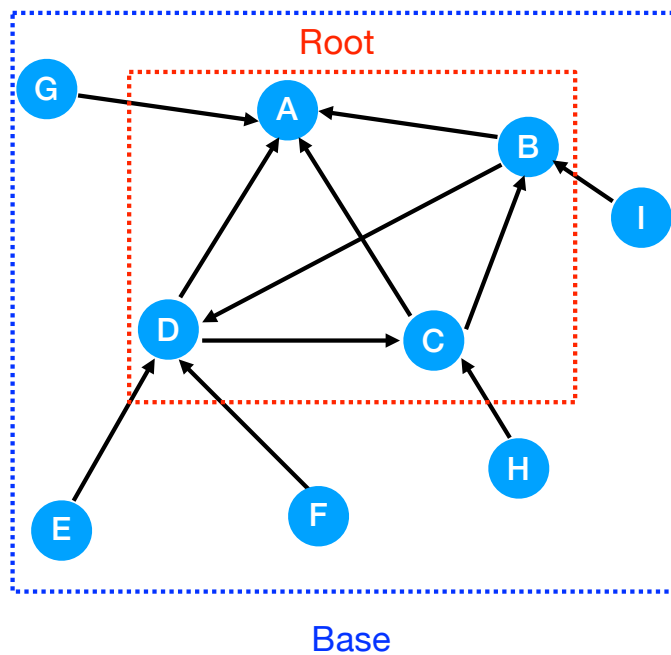
It. 2

	Auth	Hub
A	$6/11 \div 15/11$	0
B	$3/11 \div 15/11$	$7/11 \div 33/11$
C	$3/11 \div 15/11$	$6/11 \div 33/11$
D	$3/11 \div 15/11$	$6/11 \div 33/11$
E	0	$3/11 \div 33/11$
F	0	$3/11 \div 33/11$
G	0	$4/11 \div 33/11$
H	0	$2/11 \div 33/11$
I	0	$2/11 \div 33/11$

HITS Algorithm

This loop continues, until Authority and Hub scores both converges and stop changing in each iteration.

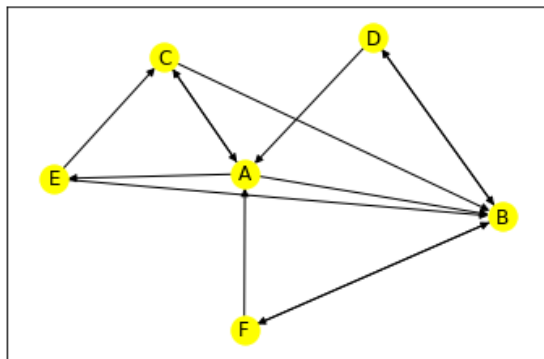
	It. 2	
	Auth	Hub
A	0.39	0
B	0.20	0.21
C	0.20	0.18
D	0.20	0.18
E	0	0.09
F	0	0.09
G	0	0.12
H	0	0.06
I	0	0.06



HITS Example

```
In [1]: import networkx as nx  
import matplotlib.pyplot as plt
```

```
In [2]: G=nx.DiGraph()  
G.add_edges_from([('A','B'),('A','C'),('D','A'),('D','B'),('B','D'),('C','A'),('C','B'),('C','B'),  
                  ('F','B'),('E','B'),('C','B'),('F','A'),('B','F'),('A','E'),('E','C'),('F','A')])  
nx.draw_networkx(G,with_labels=True,node_color='yellow')
```



HITS Example

```
In [3]: hits=nx.hits(G,max_iter=50, normalized= True)
h,a = hits
print('authority score')
print(a)
print(' \n')
print('hub score ')
print(h)
```

authority score

{'A': 0.28243511872797666, 'B': 0.454142530515933, 'C': 0.17170741178795637, 'D': -2.2060328281883135e-20, 'F': -2.2060328281883135e-20, 'E': 0.091714938968134}

hub score

{'A': 0.2019518825156947, 'B': -1.241734355099136e-20, 'C': 0.20730284712387062, 'D': 0.20730284712387062, 'F': 0.20730284712387062, 'E': 0.17613957611269343}

HITS vs Page Rank

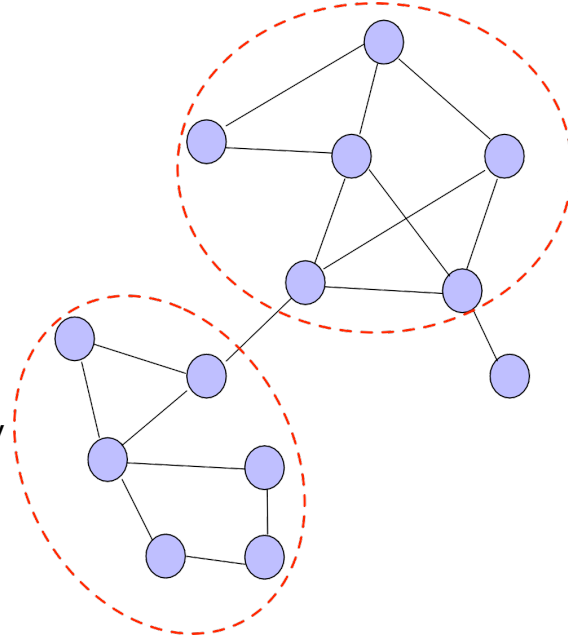
- Mathematically, the HITS algorithm can be represented as a system of equations, similar to PageRank.
- The key difference is that PageRank focuses on a single score for each page, while HITS distinguishes between **hub and authority scores**.
- HITS is particularly useful for analyzing small sets of documents or focused topics, where the distinction between hubs and authorities is meaningful. It has been applied in information retrieval and recommendation systems. However, for large-scale web search, PageRank is more commonly used due to its scalability and simplicity.

Outline

- Basic Graph Concepts
- Minimum Spanning Tree
- Shortest Path Finding
- Centrality Measurements
- Link Analysis
- **Community Detection (Louvain, Leiden)**

Community Detection

- One of the popular approaches to understanding graph networks is to identify a group of nodes inside a graph network that are tightly connected to each other, i.e. communities. This could be assumed as clustering for graph networks.
- For example, we have a network of scientists working on a special topic, but you can analyze their work and see they refer to each other's work a lot. Because they know each other, collaborate with each other, or try to support each other. In this scenario, we have a graph of authors and if they have co-authorship, they are connected to each other. Another example is to identify actors who play with each other in movies. Actors are nodes of graphs and connected to other actors if they play together in a movie.
- A community in the graph is referred to a set of nodes that are tightly connected to each other.



References

- Han, Jiawei, Pei, Jian, Tong, Hanghang. Data Mining (The Morgan Kaufmann Series in Data Management Systems). Elsevier Science. Kindle Edition.
- Hutter, Frank, Lars Kotthoff, Joaquin Vanschoren. Automated Machine Learning: Methods, Systems, Challenges (The Springer Series on Challenges in Machine Learning). Springer International Publishing. Kindle Edition.

MET CS 688

Two Clustering Use Cases

Leila Ghaedi – Fall 2024

Creator: cemagraphics | Credit: Getty Images/iStockphoto



Customer Segmentation

- In this project, we will conduct unsupervised clustering on customer records from a grocery company's database.
- Customer segmentation involves grouping customers based on similar behavior/attributes.
- The goal is to categorize customers into segments, optimizing their significance to the business.
- This segmentation allows for tailored product modifications based on unique customer needs and behaviors, facilitating the business in addressing the concerns of different customer types.

<https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis>

Customer Segmentation

Import Libraries

```
In [1]: 1 #Customer Segmentation Project
2 import mglearn
3 import numpy as np
4 import pandas as pd
5 import datetime
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.decomposition import PCA
9 import matplotlib
10 import matplotlib.pyplot as plt
11 from matplotlib import colors
12 import seaborn as sns
13 from yellowbrick.cluster import KElbowVisualizer
14 from sklearn.cluster import KMeans
15 from mpl_toolkits.mplot3d import Axes3D
16 from sklearn.cluster import AgglomerativeClustering
17 from matplotlib.colors import ListedColormap
18 from sklearn import metrics
19 import warnings
20 warnings.simplefilter(action='ignore', category=FutureWarning)
21 warnings.simplefilter(action='ignore', category=DeprecationWarning)
22 warnings.simplefilter(action='ignore', category=RuntimeWarning)
```

Customer Segmentation

Load the dataset

```
In [2]: 1 # Load the data and do the exploratory data analysis
        2 customer_data = pd.read_csv("marketing_campaign.csv", sep="\t")
        3 print("Number of records:", len(customer_data))
        4 customer_data.head()
```

Number of records: 2240

```
Out[2]:
```

	ID	Year_Birth	Education	Marital_Status	Income	Kidhome	Teenhome	Dt_Customer	Recency	MntWines	...	NumWebVisitsMonth	Ac
0	5524	1957	Graduation	Single	58138.0	0	0	04-09-2012	58	635	...	7	
1	2174	1954	Graduation	Single	46344.0	1	1	08-03-2014	38	11	...	5	
2	4141	1965	Graduation	Together	71613.0	0	0	21-08-2013	26	426	...	4	
3	6182	1984	Graduation	Together	26646.0	1	0	10-02-2014	26	11	...	6	
4	5324	1981	PhD	Married	58293.0	1	0	19-01-2014	94	173	...	5	

5 rows x 29 columns

Customer Segmentation

```
In [3]: 1 customer_data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2240 entries, 0 to 2239
Data columns (total 29 columns):
#   Column                                Non-Null Count  Dtype
---  ---                                -
0   ID                                    2240 non-null   int64
1   Year_Birth                           2240 non-null   int64
2   Education                             2240 non-null   object
3   Marital_Status                       2240 non-null   object
4   Income                               2216 non-null   float64
5   Kidhome                              2240 non-null   int64
6   Teenhome                             2240 non-null   int64
7   Dt_Customer                          2240 non-null   object
8   Recency                              2240 non-null   int64
9   MntWines                             2240 non-null   int64
10  MntFruits                            2240 non-null   int64
11  MntMeatProducts                      2240 non-null   int64
12  MntFishProducts                      2240 non-null   int64
13  MntSweetProducts                    2240 non-null   int64
14  MntGoldProds                        2240 non-null   int64
15  NumDealsPurchases                   2240 non-null   int64
16  NumWebPurchases                     2240 non-null   int64
17  NumCatalogPurchases                2240 non-null   int64
18  NumStorePurchases                   2240 non-null   int64
19  NumWebVisitsMonth                   2240 non-null   int64
20  AcceptedCmp3                        2240 non-null   int64
21  AcceptedCmp4                        2240 non-null   int64
22  AcceptedCmp5                        2240 non-null   int64
23  AcceptedCmp1                        2240 non-null   int64
24  AcceptedCmp2                        2240 non-null   int64
25  Complain                             2240 non-null   int64
26  Z_CostContact                       2240 non-null   int64
27  Z_Revenue                           2240 non-null   int64
28  Response                             2240 non-null   int64
dtypes: float64(1), int64(25), object(3)
memory usage: 507.6+ KB
```

Customer Segmentation

In [4]: 1 customer_data.describe().T

Out[4]:

	count	mean	std	min	25%	50%	75%	max
ID	2240.0	5592.159821	3246.662198	0.0	2828.25	5458.5	8427.75	11191.0
Year_Birth	2240.0	1968.805804	11.984069	1893.0	1959.00	1970.0	1977.00	1996.0
Income	2216.0	52247.251354	25173.076661	1730.0	35303.00	51381.5	68522.00	666666.0
Kidhome	2240.0	0.444196	0.538398	0.0	0.00	0.0	1.00	2.0
Teenhome	2240.0	0.506250	0.544538	0.0	0.00	0.0	1.00	2.0
Recency	2240.0	49.109375	28.962453	0.0	24.00	49.0	74.00	99.0
MntWines	2240.0	303.935714	336.597393	0.0	23.75	173.5	504.25	1493.0
MntFruits	2240.0	26.302232	39.773434	0.0	1.00	8.0	33.00	199.0
MntMeatProducts	2240.0	166.950000	225.715373	0.0	16.00	67.0	232.00	1725.0
MntFishProducts	2240.0	37.525446	54.628979	0.0	3.00	12.0	50.00	259.0
MntSweetProducts	2240.0	27.062946	41.280498	0.0	1.00	8.0	33.00	263.0
MntGoldProds	2240.0	44.021875	52.167439	0.0	9.00	24.0	56.00	362.0
NumDealsPurchases	2240.0	2.325000	1.932238	0.0	1.00	2.0	3.00	15.0
NumWebPurchases	2240.0	4.084821	2.778714	0.0	2.00	4.0	6.00	27.0
NumCatalogPurchases	2240.0	2.662054	2.923101	0.0	0.00	2.0	4.00	28.0
NumStorePurchases	2240.0	5.790179	3.250958	0.0	3.00	5.0	8.00	13.0
NumWebVisitsMonth	2240.0	5.316518	2.426645	0.0	3.00	6.0	7.00	20.0
AcceptedCmp3	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.00	1.0
AcceptedCmp4	2240.0	0.074554	0.262728	0.0	0.00	0.0	0.00	1.0
AcceptedCmp5	2240.0	0.072768	0.259813	0.0	0.00	0.0	0.00	1.0
AcceptedCmp1	2240.0	0.064286	0.245316	0.0	0.00	0.0	0.00	1.0
AcceptedCmp2	2240.0	0.013393	0.114976	0.0	0.00	0.0	0.00	1.0
Complain	2240.0	0.009375	0.096391	0.0	0.00	0.0	0.00	1.0
Z_CostContact	2240.0	3.000000	0.000000	3.0	3.00	3.0	3.00	3.0
Z_Revenue	2240.0	11.000000	0.000000	11.0	11.00	11.0	11.00	11.0
Response	2240.0	0.149107	0.356274	0.0	0.00	0.0	0.00	1.0

Customer Segmentation Data

- The data consists of 2240 records, where each record has 29 attributes.
- Attributes:
 1. Personal Information (ID, Year of Birth, Education, Marital Status,...)
 2. Product (MntWines, MntFruits, MntMeatProducts,...)
 3. Promotion (NumDealsPurchases, AcceptedCmp1, AcceptedCmp2, ...)
 4. Channel (NumWebPurchases, NumCatalogPurchases,...)

Customer Segmentation

Personal Information:

ID: Customer's unique identifier

Year_Birth: Customer's birth year

Education: Customer's education level

Marital_Status: Customer's marital status

Income: Customer's yearly household income

Kidhome: Number of children in customer's household

Teenhome: Number of teenagers in customer's household

Dt_Customer: Date of customer's enrollment with the company

Recency: Number of days since customer's last purchase

Complain: 1 if the customer complained in the last 2 years, 0 otherwise

Customer Segmentation

Products:

MntWines: Amount spent on wine in last 2 years

MntFruits: Amount spent on fruits in last 2 years

MntMeatProducts: Amount spent on meat in last 2 years

MntFishProducts: Amount spent on fish in last 2 years

MntSweetProducts: Amount spent on sweets in last 2 years

MntGoldProds: Amount spent on gold in last 2 years

Customer Segmentation

Promotion:

NumDealsPurchases: Number of purchases made with a discount

AcceptedCmp1: 1 if customer accepted the offer in the 1st campaign, 0 otherwise

AcceptedCmp2: 1 if customer accepted the offer in the 2nd campaign, 0 otherwise

AcceptedCmp3: 1 if customer accepted the offer in the 3rd campaign, 0 otherwise

AcceptedCmp4: 1 if customer accepted the offer in the 4th campaign, 0 otherwise

AcceptedCmp5: 1 if customer accepted the offer in the 5th campaign, 0 otherwise

Response: 1 if customer accepted the offer in the last campaign, 0 otherwise

Customer Segmentation

Channel:

- NumWebPurchases: Number of purchases made through the company's website
- NumCatalogPurchases: Number of purchases made using a catalogue
- NumStorePurchases: Number of purchases made directly in stores
- NumWebVisitsMonth: Number of visits to company's website in the last month

Customer Segmentation

Categorical Variables

```
In [5]: 1 # Categorical Variable levels|
        2 print("Marital_Status:\n", customer_data["Marital_Status"].value_counts(), "\n")
        3 print("Education:\n", customer_data["Education"].value_counts())
```

```
Marital_Status:
Marital_Status
Married      864
Together     580
Single       480
Divorced     232
Widow        77
Alone         3
Absurd        2
YOLO          2
Name: count, dtype: int64
```

```
Education:
Education
Graduation   1127
PhD           486
Master       370
2n Cycle     203
Basic         54
Name: count, dtype: int64
```


Customer Segmentation

Missing Data

- There are 24 missing data points in Income field.
- Since the missing data is just few records and in one attribute, we can drop the missing records.

```
In [6]: 1 # Drop missing data points
        2 customer_data = customer_data.dropna()
        3 print("The total number of recrds after removing the rows with missing values:", len(customer_data))
```

The total number of recrds after removing the rows with missing values: 2216

Customer Segmentation Formatting

- Dt_Customer that indicates the date a customer joined the database is not parsed as DateTime.
- We need to encode categorical and binary features as well.

```
In [7]: 1 # date time formatting  
        2 customer_data["Dt_Customer"] = pd.to_datetime(customer_data["Dt_Customer"], format='%d-%m-%Y')
```

Customer Segmentation

Feature Engineering

- Create a “Tenure” field shows the number of days the customers started to shop in the store relative to the last recorded date.
- Extract “Age” at 2012 based on Year_Birth.
- Create feature "Spent" indicating the total amount spent by the customer in all categories over the span of two years.
- Create another feature "Living_With" out of "Marital_Status" to extract the living situation of couples.
- Create a feature "Children" to indicate total children in a household that is, kids and teenagers.
- Creating feature indicating "Family_Size"
- Create a feature "Parent" to indicate parenthood status
- Create three categories in the "Education" by simplifying its value counts.
- Drop some of the redundant features.

Customer Segmentation

Feature Engineering

In [8]: 1 *# Feature Engineering Steps*

```
In [9]: 1 dates = []
2 for i in customer_data["Dt_Customer"]:
3     i = i.date()
4     dates.append(i)
5 #Dates of the newest and oldest recorded customer
6 print("The newest customer's enrolment date in therecords:",max(dates))
7 print("The oldest customer's enrolment date in the records:",min(dates))
```

The newest customer's enrolment date in therecords: 2014-06-29
The oldest customer's enrolment date in the records: 2012-07-30

```
In [10]: 1 # Create a feature "Tenure"
2 days = []
3 d1 = max(dates) # compare with the newest customer date
4 for i in dates:
5     delta = d1 - i
6     days.append(delta.days)
7 customer_data["Tenure"] = days
8 customer_data["Tenure"] = pd.to_numeric(customer_data["Tenure"], errors="coerce")
9
```

Customer Segmentation

Feature Engineering

```
In [11]: 1 #Create "Age" at 2012
          2 customer_data["Age"] = 2012- customer_data["Year_Birth"]
          3
```

```
In [12]: 1 #Remove outliers by setting a cap on Age and income.
          2
          3 customer_data = customer_data[(customer_data["Age"]<100)]
          4 customer_data = customer_data[(customer_data["Income"]<600000)]
          5 print("The total number of recrds after removing the routliers:", len(customer_data))
```

The total number of recrds after removing the routliers: 2212

```
In [13]: 1 #Total spend on all items
          2 customer_data["Spent"] = customer_data["MntWines"]+ \
          3                     customer_data["MntFruits"]+ \
          4                     customer_data["MntMeatProducts"]+ \
          5                     customer_data["MntFishProducts"]+ \
          6                     customer_data["MntSweetProducts"]+ \
          7                     customer_data["MntGoldProds"]
          8
          9
```

Customer Segmentation

Feature Engineering

```
In [14]: 1 # living situation by marital status "Alone" vs "Partner"
2 customer_data["Living_With"] = customer_data["Marital_Status"].\
3     replace({"Married": "Partner", "Together": "Partner", \
4             "Absurd": "Alone", "Widow": "Alone", "YOLO": "Alone", \
5             "Divorced": "Alone", "Single": "Alone", })

In [15]: 1 #total children living in the household
2 customer_data["Children"] = customer_data["Kidhome"] + customer_data["Teenhome"]
3
4 # total members in the household
5 customer_data["Family_Size"] = customer_data["Living_With"].\
6     replace({"Alone": 1, "Partner": 2}) + customer_data["Children"]
7 # parenthood
8 customer_data["Parent"] = np.where(customer_data.Children > 0, 1, 0)

In [16]: 1 #Segmenting education levels in three groups
2 customer_data["Education"] = customer_data["Education"].\
3     replace({"Basic": "Undergraduate", "2n Cycle": "Undergraduate", \
4             "Graduation": "Graduate", "Master": "Postgraduate", \
5             "PhD": "Postgraduate"})
```

Customer Segmentation Feature Engineering

```
In [17]: 1 # drop the redundant features
          2 to_drop = ["Marital_Status", "Year_Birth", "ID", "Z_CostContact", "Z_Revenue", "Dt_Customer"]
          3 customer_data = customer_data.drop(to_drop, axis=1)
```

Customer Segmentation-Feature Engineering

In [18]: 1 customer_data.describe().T

Out[18]:

	count	mean	std	min	25%	50%	75%	max
Income	2212.0	51958.810579	21527.278844	1730.0	35233.5	51371.0	68487.00	162397.0
Kidhome	2212.0	0.441682	0.536955	0.0	0.0	0.0	1.00	2.0
Teenhome	2212.0	0.505877	0.544253	0.0	0.0	0.0	1.00	2.0
Recency	2212.0	49.019439	28.943121	0.0	24.0	49.0	74.00	99.0
MntWines	2212.0	305.287523	337.322940	0.0	24.0	175.5	505.00	1493.0
MntFruits	2212.0	26.329566	39.744052	0.0	2.0	8.0	33.00	199.0
MntMeatProducts	2212.0	167.029837	224.254493	0.0	16.0	68.0	232.25	1725.0
MntFishProducts	2212.0	37.648734	54.772033	0.0	3.0	12.0	50.00	259.0
MntSweetProducts	2212.0	27.046564	41.090991	0.0	1.0	8.0	33.00	262.0
MntGoldProds	2212.0	43.925859	51.706981	0.0	9.0	24.5	56.00	321.0
NumDealsPurchases	2212.0	2.324593	1.924507	0.0	1.0	2.0	3.00	15.0
NumWebPurchases	2212.0	4.088156	2.742187	0.0	2.0	4.0	6.00	27.0
NumCatalogPurchases	2212.0	2.672242	2.927542	0.0	0.0	2.0	4.00	28.0
NumStorePurchases	2212.0	5.806510	3.250939	0.0	3.0	5.0	8.00	13.0
NumWebVisitsMonth	2212.0	5.321429	2.425597	0.0	3.0	6.0	7.00	20.0
AcceptedCmp3	2212.0	0.073689	0.261323	0.0	0.0	0.0	0.00	1.0
AcceptedCmp4	2212.0	0.074141	0.262060	0.0	0.0	0.0	0.00	1.0
AcceptedCmp5	2212.0	0.072785	0.259842	0.0	0.0	0.0	0.00	1.0
AcceptedCmp1	2212.0	0.064195	0.245156	0.0	0.0	0.0	0.00	1.0
AcceptedCmp2	2212.0	0.013562	0.115691	0.0	0.0	0.0	0.00	1.0
Complain	2212.0	0.009042	0.094678	0.0	0.0	0.0	0.00	1.0
Response	2212.0	0.150542	0.357683	0.0	0.0	0.0	0.00	1.0
Tenure	2212.0	353.714286	202.494886	0.0	180.0	356.0	529.00	699.0
Age	2212.0	43.086347	11.701599	16.0	35.0	42.0	53.00	72.0
Spent	2212.0	607.268083	602.513364	5.0	69.0	397.0	1048.00	2525.0
Children	2212.0	0.947559	0.749466	0.0	0.0	1.0	1.00	3.0
Family_Size	2212.0	2.593128	0.908236	1.0	2.0	3.0	3.00	5.0
Parent	2212.0	0.714286	0.451856	0.0	0.0	1.0	1.00	1.0

Customer Segmentation-Feature Engineering

```
In [19]: 1 # only numerical values
          2 to_drop = ["Education", "Living_With"]
          3 customer_data_num= customer_data.drop(to_drop, axis=1)
          4
          5 #correlation matrix
          6 corrmatrix= customer_data_num.corr()
          7 plt.figure(figsize=(20,20))
          8 sns.heatmap(corrmatrix,annot=True, center=0)
```

Customer Segmentation

Data Pre-Processing

- The following steps are applied to preprocess the data:
 - Label encoding the categorical features
 - Scaling the features using the standard scaler
 - Creating a subset dataframe for dimensionality reduction

Customer Segmentation-Data Pre-Processing

```
In [20]: 1 # Save a copy of the data for profiling before encoding categories
2 customer_data_copy=customer_data
3 # Encode all categorical features
4 #Label Encoding the object dtypes.
5 LE=LabelEncoder()
6 for i in ['Education', 'Living_With']:
7     customer_data[i]=customer_data[[i]].apply(LE.fit_transform)
8
```

```
In [21]: 1 # Scale and save in a new data frame scaled_df
2 scaler = StandardScaler()
3 scaler.fit(customer_data)
4 scaled_df = pd.DataFrame(scaler.transform(customer_data),columns= customer_data.columns )
5 scaled_df.head()
```

```
Out[21]:
```

	Education	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFishProducts	MntSweetProducts	...	Age
0	-0.893586	0.287105	-0.822754	-0.929699	0.310353	0.977660	1.552041	1.690293	2.453472	1.483713	...	29
1	-0.893586	-0.260882	1.040021	0.908097	-0.380813	-0.872618	-0.637461	-0.718230	-0.651004	-0.634019	...	29
2	-0.893586	0.913196	-0.822754	-0.929699	-0.795514	0.357935	0.570540	-0.178542	1.339513	-0.147184	...	29
3	-0.893586	-1.176114	1.040021	-0.929699	-0.795514	-0.872618	-0.561961	-0.655787	-0.504911	-0.585335	...	29
4	0.571657	0.294307	1.040021	-0.929699	1.554453	-0.392257	0.419540	-0.218684	0.152508	-0.001133	...	29

5 rows x 30 columns

Customer Segmentation-Data Pre-Processing

PCA

- In this problem, there are 30 factors based on which the final clustering will be done.
- These factors are basically attributes or features.
- Many of these features are correlated, and hence redundant.
- This is why we will be performing dimensionality reduction on the features before feeding them to a clustering model.

Customer Segmentation-Data Pre-Processing

PCA

```
In [22]: 1 # PCA for dimensionality reduction
          2 #Initiating PCA to reduce dimentions aka features to 3
          3 pca = PCA(n_components=3)
          4 pca.fit(scaled_df)
          5 PCA_df = pd.DataFrame(pca.transform(scaled_df), columns=["column1","column2", "column3" ])
          6 PCA_df.describe().T
```

Out[22]:

	count	mean	std	min	25%	50%	75%	max
column1	2212.0	-2.569775e-17	2.960584	-5.924200	-2.556958	-0.861992	2.264132	8.537421
column2	2212.0	4.497106e-17	1.717093	-4.159480	-1.348848	-0.159602	1.248615	6.064053
column3	2212.0	1.284887e-17	1.389386	-3.186204	-0.786302	-0.163359	0.411128	8.260273

Customer Segmentation- Clustering

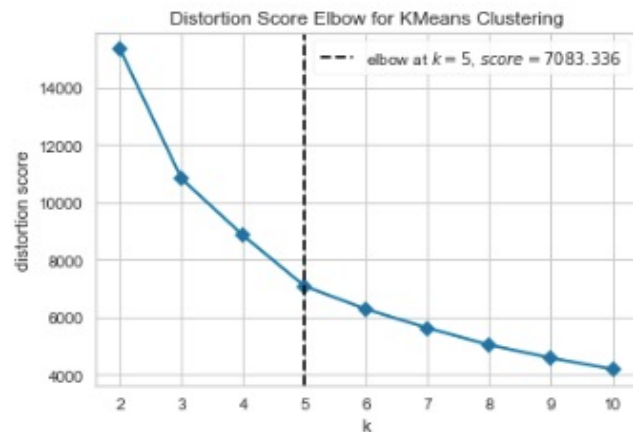
1. Elbow Method to determine the number of clusters to be formed
2. Clustering using Agglomerative Clustering
3. Examining the clusters formed visually

Customer Segmentation- Clustering

Elbow Method

```
In [23]: 1 # Elbow method to find numbers of clusters (based on PCA_df)
2 print('Elbow Method for PCA data set:')
3 Elbow_M = KElbowVisualizer(KMeans(random_state=42),k=10, timings=False)
4 Elbow_M.fit(PCA_df)
5 Elbow_M.show()
```

Elbow Method for PCA data set:



```
Out[23]: <Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>
```

Customer Segmentation- Clustering

Agglomerative Clustering

In [24]:

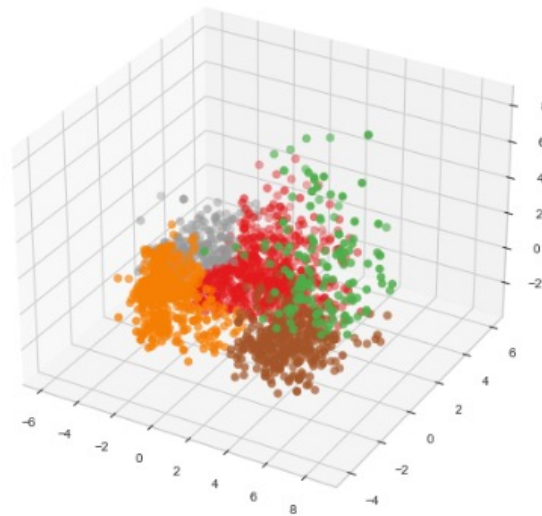
```
1 #Initiating the Agglomerative Clustering model
2 AC = AgglomerativeClustering(n_clusters=5)
3 # fit model and predict clusters
4 yhat_AC = AC.fit_predict(PCA_df)
5 PCA_df["Clusters"] = yhat_AC
6 #Adding the Clusters feature to the original dataframe.
7 customer_data["Clusters"] = yhat_AC
8 customer_data_copy["Clusters"] = yhat_AC
```


Customer Segmentation- Clustering

Agglomerative Clustering

```
In [25]: 1 from matplotlib import cm
2 tab20_5 = cm.get_cmap("Set1", lut = 5)
3 x =PCA_df["column1"]
4 y =PCA_df["column2"]
5 z =PCA_df["column3"]
6 #Plotting the clusters
7 fig = plt.figure(figsize=(10,8))
8 ax = plt.subplot(111, projection='3d', label="bla")
9 ax.scatter(x, y, z, s=40, c=PCA_df["Clusters"], marker='o' , cmap=tab20_5)
10 ax.set_title("The Plot Of The Clusters")
11 plt.show()
```

The Plot Of The Clusters

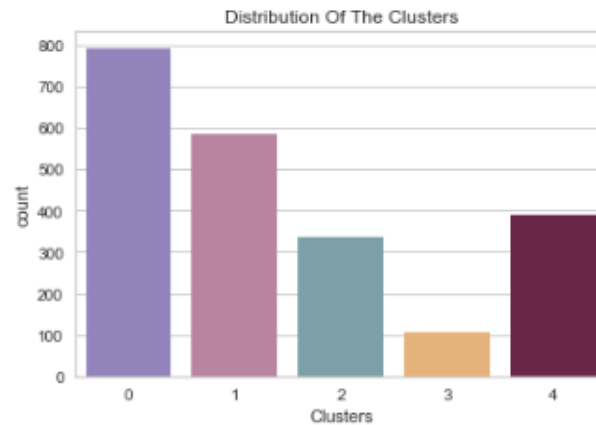


Customer Segmentation- Clustering

Understanding the clusters

In [26]:

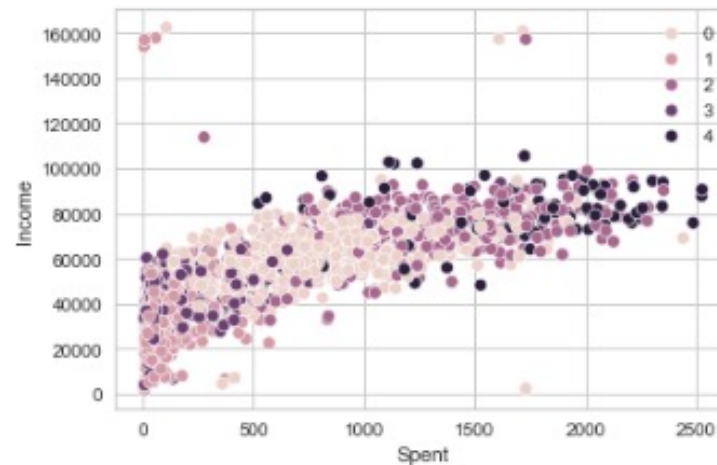
```
1 # countplot of clusters
2 #Plotting countplot of clusters
3 pal = ["#8E7CC3", "#C27BA0", "#76A5AF", "#f6b26b", "#741b47"]
4
5
6 pl = sns.countplot(x=customer_data["Clusters"], palette= pal)
7 pl.set_title("Distribution Of The Clusters")
8 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

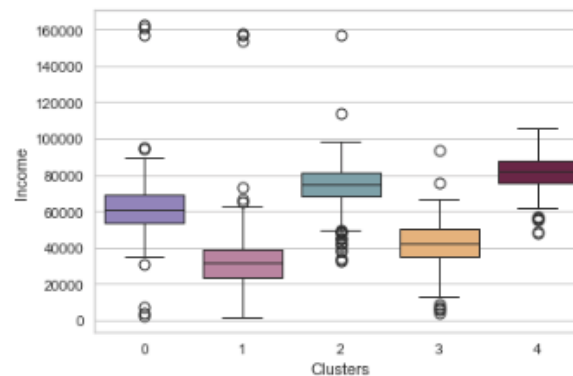
```
In [27]: 1 l = sns.scatterplot(data = customer_data,x=customer_data["Spent"], \
2                               y=customer_data["Income"],hue=customer_data["Clusters"])
3 pl.set_title("Cluster's Profile Based On Income And Spending")
4 plt.legend()
5 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

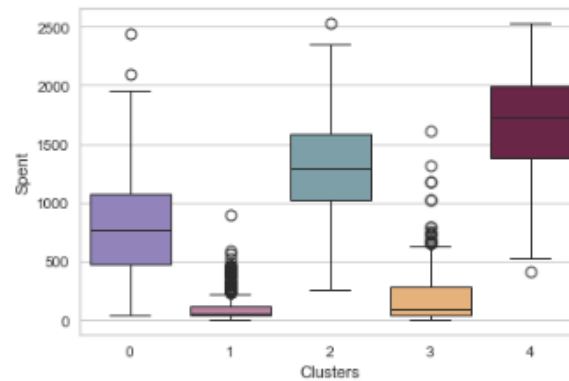
```
In [29]: 1 plt.figure()  
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Income"], palette=pal)  
3 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

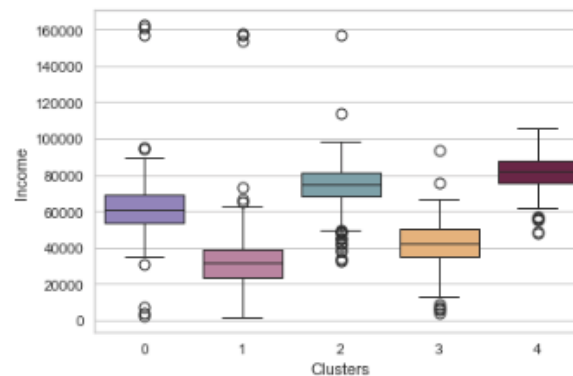
```
In [28]: 1 plt.figure()
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Spent"], palette=pal)
3 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

```
In [29]: 1 plt.figure()  
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Income"], palette=pal)  
3 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

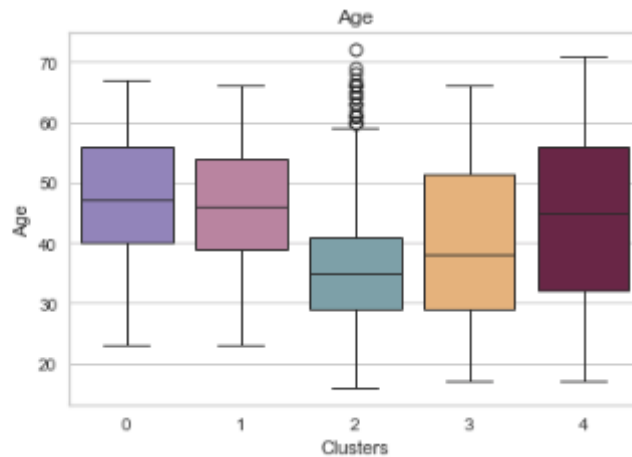
- Observation: clusters patterns in terms of spend and income are similar

Cluster	Median Income	Median Spend	Spend/income
0	\$65000	\$976	1.50%
1	\$45000	\$182	0.40%
2	\$75000	\$1285	1.71%
3	\$29000	\$57	0.20%
4	\$83000	\$1772	2.13%

Customer Segmentation- Clustering

Understanding the clusters

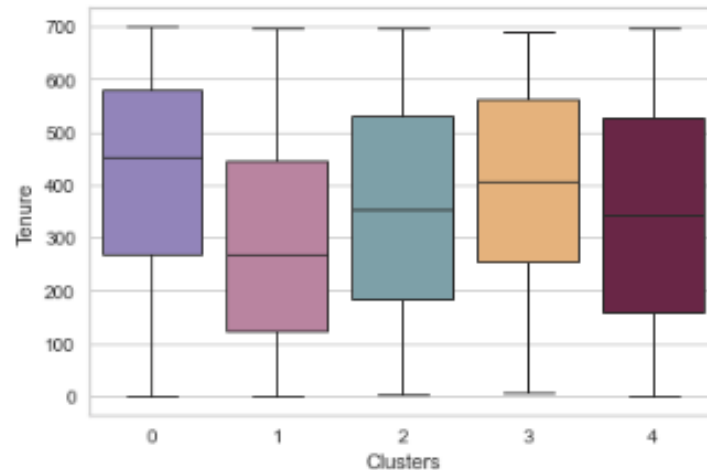
```
In [30]: 1 plt.figure()  
2 plt.figure()  
3 pl=sns.boxplot(y=customer_data["Age"],x=customer_data["Clusters"], palette=pal)  
4 pl.set_title("Age")  
5 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

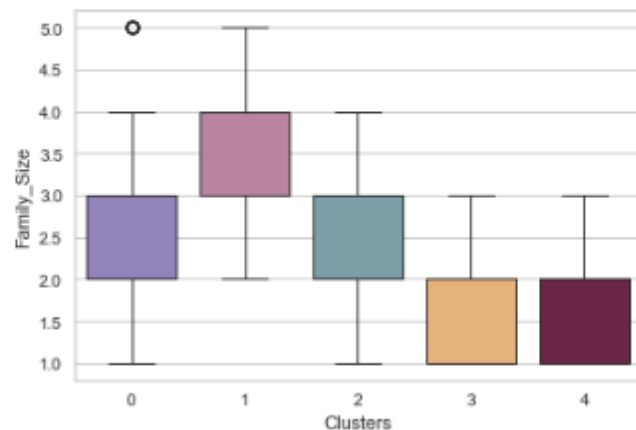
```
In [31]: 1 plt.figure()  
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Tenure"], palette=pal)  
3 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

```
In [32]: 1 plt.figure()  
2 pl=sns.boxplot(x=customer_data["Clusters"], y=customer_data["Family_Size"], palette=pal)  
3 plt.show()
```

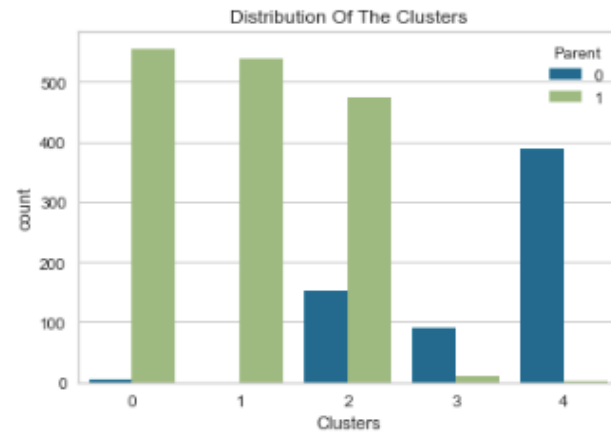


Customer Segmentation- Clustering

Understanding the clusters

In [33]:

```
1 pl = sns.countplot(x=customer_data["Clusters"], hue=customer_data["Parent"])
2 pl.set_title("Distribution Of The Clusters")
3 plt.show()
```



Customer Segmentation- Clustering

Understanding the clusters

Cluster	Characteristics
0	Highly likely to be parents, High Spender, Large Family, Highest Median Tenure (454 days)
1	Highly likely to be parents, Low Spender, Largest Family Size, Lowest Median Tenure (296 days)
2	Lowest Spender, Large Family
3	Highly unlikely to be parents, Highest Spender, High Spend on wine, Small Family
4	Highly unlikely to be parents, High Spender, Small Family

Patient Segmentation

- In this project, we will conduct unsupervised clustering on patients records from an insurance database.
- Patients' segmentation involves grouping them based on similar behavior/attributes.
- The goal is to categorize patients into segments, optimizing their significance to the insurance company.

Patient Segmentation

Import Libraries

```
In [1]: 1 #Patients Segmentation Project
2 import numpy as np
3 import pandas as pd
4 from plotly import tools
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7 from string import ascii_letters
8 from scipy.stats import norm
9 from scipy.stats import skew
10 from scipy.stats import pearsonr
11 from scipy import stats
12 import statsmodels.api as sm
13 import plotly.graph_objs as go
14 from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot
15 from statsmodels.sandbox.regression.predstd import wls_prediction_std
16 from sklearn.preprocessing import LabelEncoder
17 from sklearn.preprocessing import StandardScaler
18 from yellowbrick.cluster import KElbowVisualizer
19 from sklearn.cluster import KMeans
20 import warnings
21 warnings.filterwarnings("ignore")
```

Patient Segmentation

Load the dataset

```
In [2]: 1 # Load the data and do the exploratory data analysis
        2 patient_data = pd.read_csv("patients.csv", sep=",")
        3 patient_data_original=patient_data
        4 print("Number of records:", len(patient_data))
        5 patient_data.head()
```

Number of records: 1338

Out[2]:

	age	sex	bmi	children	smoker	region	charges
0	19	female	27.900	0	yes	southwest	16884.92400
1	18	male	33.770	1	no	southeast	1725.55230
2	28	male	33.000	3	no	southeast	4449.46200
3	33	male	22.705	0	no	northwest	21984.47061
4	32	male	28.880	0	no	northwest	3866.85520

Patient Segmentation

```
In [3]: 1 patient_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 1338 entries, 0 to 1337  
Data columns (total 7 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   age         1338 non-null   int64  
1   sex         1338 non-null   object  
2   bmi         1338 non-null   float64  
3   children    1338 non-null   int64  
4   smoker      1338 non-null   object  
5   region      1338 non-null   object  
6   charges     1338 non-null   float64  
dtypes: float64(2), int64(2), object(3)  
memory usage: 73.3+ KB
```


Patient Segmentation

In [4]: 1 patient_data.describe().T

Out[4]:

	count	mean	std	min	25%	50%	75%	max
age	1338.0	39.207025	14.049960	18.0000	27.00000	39.000	51.000000	64.00000
bmi	1338.0	30.663397	6.098187	15.9600	26.29625	30.400	34.693750	53.13000
children	1338.0	1.094918	1.205493	0.0000	0.00000	1.000	2.000000	5.00000
charges	1338.0	13270.422265	12110.011237	1121.8739	4740.28715	9382.033	16639.912515	63770.42801

Patient Segmentation

```
In [5]: 1 # Categorical Variable levels
        2 print("Smoker:\n", patient_data["sex"].value_counts(), "\n")
        3 print("Smoker:\n", patient_data["smoker"].value_counts(), "\n")
        4 print("Region:\n", patient_data["region"].value_counts())
```

```
Smoker:
sex
male      676
female    662
Name: count, dtype: int64
```

```
Smoker:
smoker
no       1064
yes       274
Name: count, dtype: int64
```

```
Region:
region
southeast    364
southwest    325
northwest    325
northeast    324
Name: count, dtype: int64
```

Patient Segmentation

```
In [6]: 1 # Drop missing data points
        2 patient_data = patient_data.dropna()
        3 print("The total number of recrds after removing the rows with missing values:", len(patient_data))
```

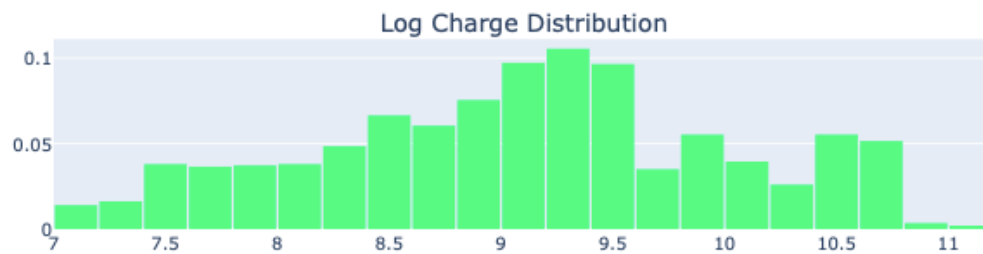
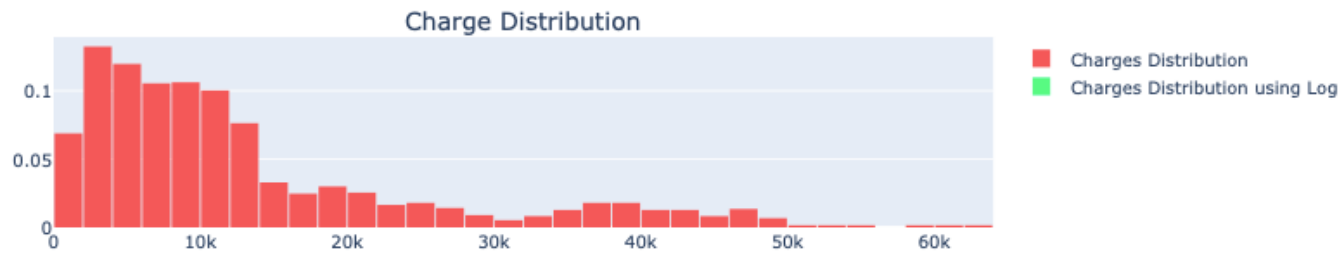
The total number of recrds after removing the rows with missing values: 1338

Patient Segmentation

```
In [7]: 1 #EDA (Exploratory Data Analysis)
2
3 # Determine the distribution of charge
4 charge_dist = patient_data["charges"].values
5 logcharge = np.log(patient_data["charges"])
6
7 trace0 = go.Histogram(
8     x=charge_dist,
9     histnorm='probability',
10    name="Charges Distribution",
11    marker = dict(
12        color = '#FA5858',
13    )
14 )
15 trace1 = go.Histogram(
16     x=logcharge,
17     histnorm='probability',
18     name="Charges Distribution using Log",
19     marker = dict(
20         color = '#58FA82',
21     )
22 )
23
24 fig = tools.make_subplots(rows=2, cols=1,
25                          subplot_titles=('Charge Distribution', 'Log Charge Distribution'),
26                          print_grid=False)
27
28 fig.append_trace(trace0, 1, 1)
29 fig.append_trace(trace1, 2, 1)
30
31 fig['layout'].update(showlegend=True, title='Charge Distribution', bargap=0.05)
32 iplot(fig, filename='custom-sized-subplot-with-subplot-titles')
```

Patient Segmentation

Charge Distribution

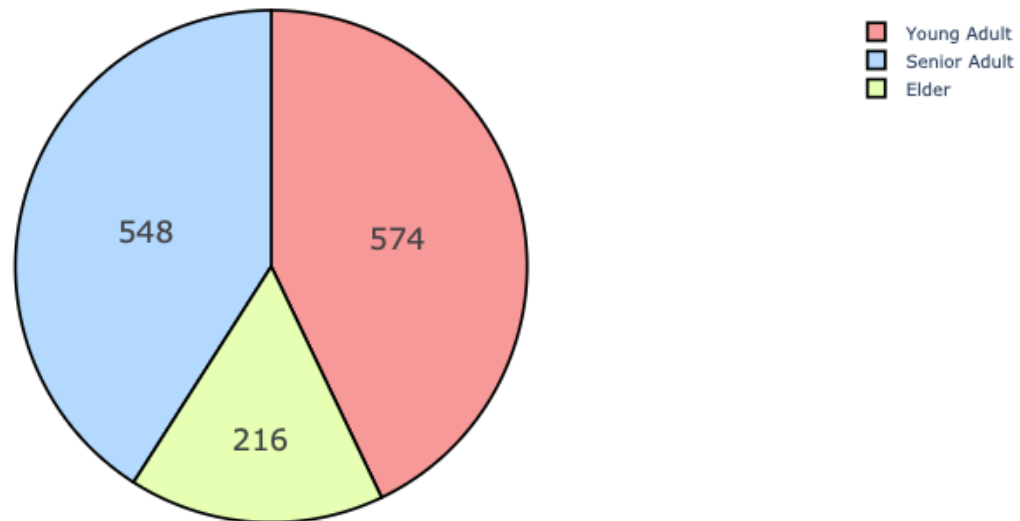


Patient Segmentation

```
In [8]: 1 patient_data['age_cat'] = np.nan
2 lst = [patient_data]
3
4 for col in lst:
5     col.loc[(col['age'] >= 18) & (col['age'] <= 35), 'age_cat'] = 'Young Adult'
6     col.loc[(col['age'] > 35) & (col['age'] <= 55), 'age_cat'] = 'Senior Adult'
7     col.loc[col['age'] > 55, 'age_cat'] = 'Elder'
8
9 labels = patient_data["age_cat"].unique().tolist()
10 amount = patient_data["age_cat"].value_counts().tolist()
11
12 colors = ["#ff9999", "#b3d9ff", "#e6ffb3"]
13
14 trace = go.Pie(labels=labels, values=amount,
15               hoverinfo='label+percent', textinfo='value',
16               textfont=dict(size=20),
17               marker=dict(colors=colors,
18                           line=dict(color='#000000', width=2)))
19
20 data = [trace]
21 layout = go.Layout(title="Amount by Age Category")
22 fig = go.Figure(data=data, layout=layout)
23 iplot(fig, filename='basic_pie_chart')
```

Patient Segmentation

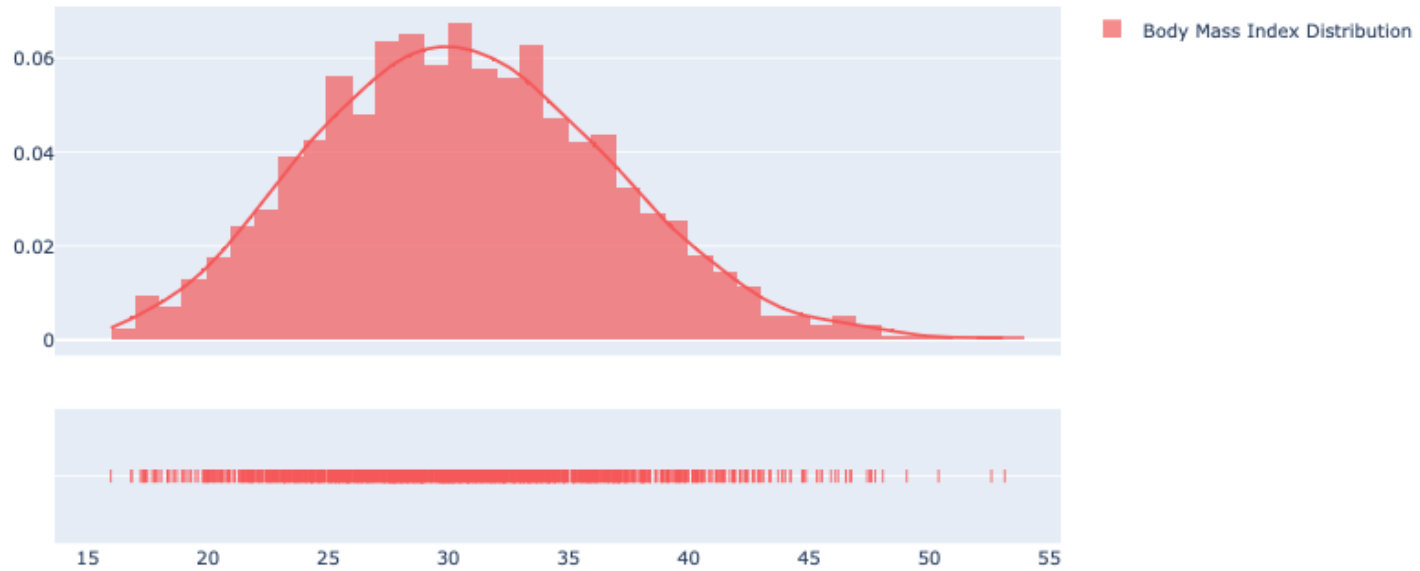
Amount by Age Category



Patient Segmentation

```
In [9]: 1 # BMI Distribution
2 import plotly.figure_factory as ff
3 bmi = [patient_data["bmi"].values.tolist()]
4 group_labels = ['Body Mass Index Distribution']
5
6 colors = ['#FA5858']
7
8 fig = ff.create_distplot(bmi, group_labels, colors=colors)
9 # Add title
10
11 iplot(fig, filename='Basic Distplot')
```


Patient Segmentation

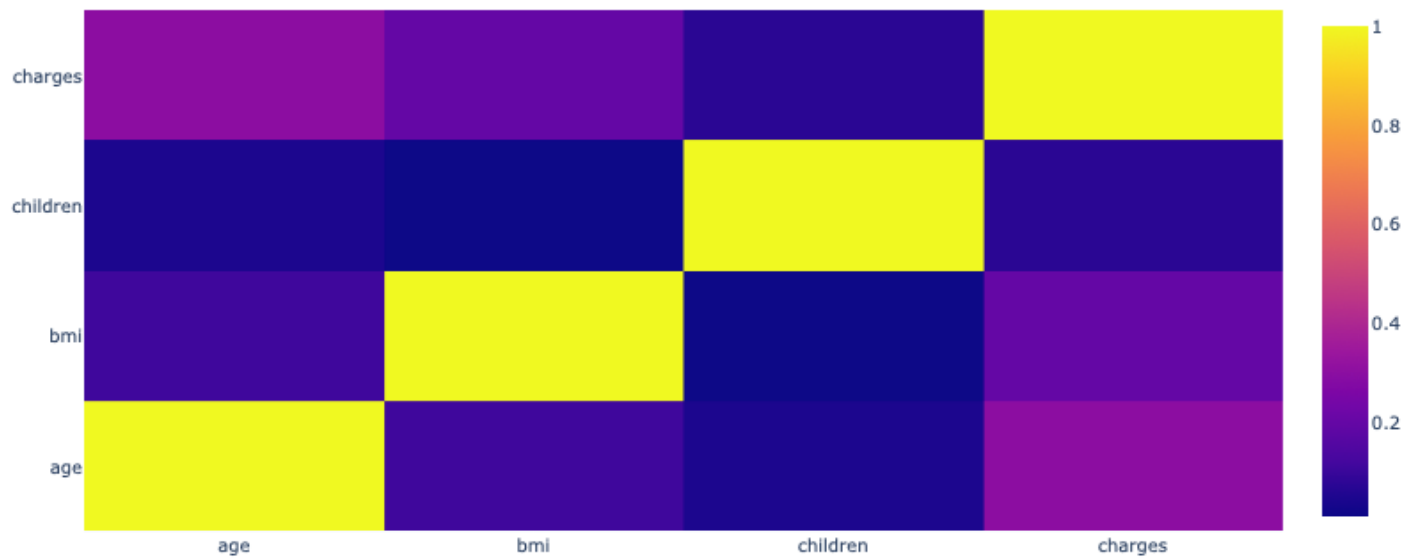


Patient Segmentation

```
In [10]: 1 # Correlation heatmap of numerical fields
2 df=patient_data_original.drop(columns=["sex", "smoker","region"])
3 corr = df.corr()
4
5 hm = go.Heatmap(
6     z=corr.values,
7     x=corr.index.values.tolist(),
8     y=corr.index.values.tolist()
9 )
10
11
12 data = [hm]
13 layout = go.Layout(title="Correlation Heatmap")
14
15 fig = dict(data=data, layout=layout)
16 iplot(fig, filename='labelled-heatmap')
```

Patient Segmentation

Correlation Heatmap



Patient Segmentation

```
In [11]: 1 # Clustering Pre-Processing Steps
          2 LE=LabelEncoder()
          3 LE=LabelEncoder()
          4 for i in ['age_cat', 'sex', 'smoker', 'region']:
          5     patient_data[i]=patient_data[[i]].apply(LE.fit_transform)
          6
```

```
In [12]: 1 # Scale and save in a new data frame scaled_df
          2 patient_data=patient_data.drop(columns=["charges", "age"])
          3 scaler = StandardScaler()
          4 scaler.fit(patient_data)
          5 scaled_df = pd.DataFrame(scaler.transform(patient_data),columns= patient_data.columns )
          6 scaled_df.head()
```

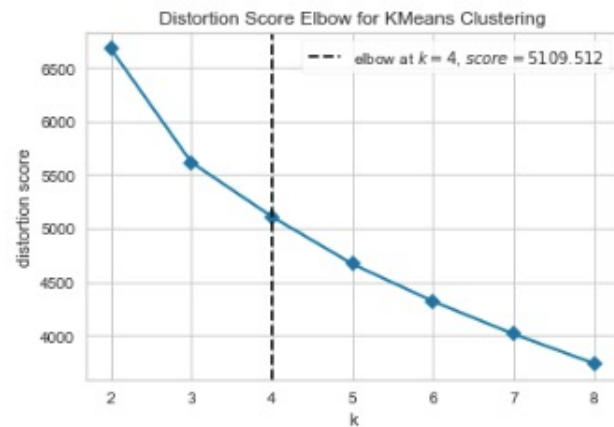
Out[12]:

	sex	bmi	children	smoker	region	age_cat
0	-1.010519	-0.453320	-0.908614	1.970587	1.343905	1.016838
1	0.989591	0.509621	-0.078767	-0.507463	0.438495	1.016838
2	0.989591	0.383307	1.580926	-0.507463	0.438495	1.016838
3	0.989591	-1.305531	-0.908614	-0.507463	-0.466915	1.016838
4	0.989591	-0.292556	-0.908614	-0.507463	-0.466915	1.016838

Patient Segmentation

```
In [13]: 1 # Elbow method to find numbers of clusters (based on scaled_df)
2 print('Elbow Method for PCA data set:')
3 Elbow_M = KElbowVisualizer(KMeans(random_state=42),k=8, timings=False)
4 Elbow_M.fit(scaled_df)
5 Elbow_M.show()
```

Elbow Method for PCA data set:



```
Out[13]: <Axes: title={'center': 'Distortion Score Elbow for KMeans Clustering'}, xlabel='k', ylabel='distortion score'>
```

Patient Segmentation

```
In [14]: 1 #Initiating the Agglomerative Clustering model
          2 AC = AgglomerativeClustering(n_clusters=4)
          3 # fit model and predict clusters
          4 yhat_AC = AC.fit_predict(scaled_df)
          5 scaled_df["Clusters"] = yhat_AC
          6 #Adding the Clusters feature to the original dataframe.
          7 scaled_df["Clusters"] = yhat_AC
          8 patient_data_original["Clusters"] = yhat_AC
```

Patient Segmentation

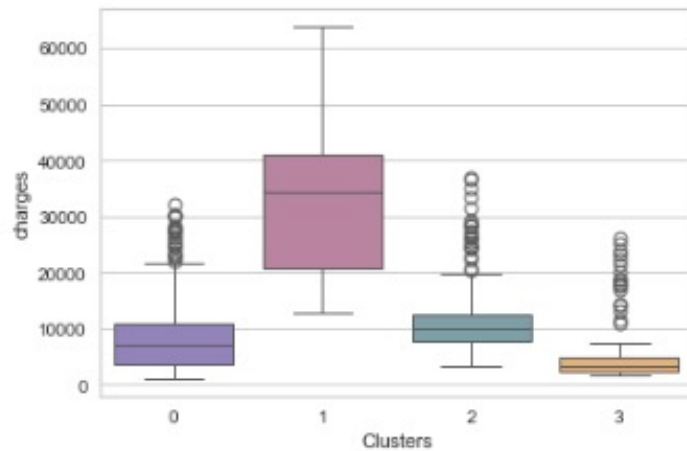
```
In [15]: 1 # countplot of clusters
2 #Plotting countplot of clusters
3 pal = ["#8E7CC3", "#C27BA0", "#76A5AF", "#f6b26b", "#741b47"]
4
5
6 pl = sns.countplot(x=patient_data_original["Clusters"], palette= pal)
7 pl.set_title("Distribution Of The Clusters")
8 plt.show()
```



Patient Segmentation

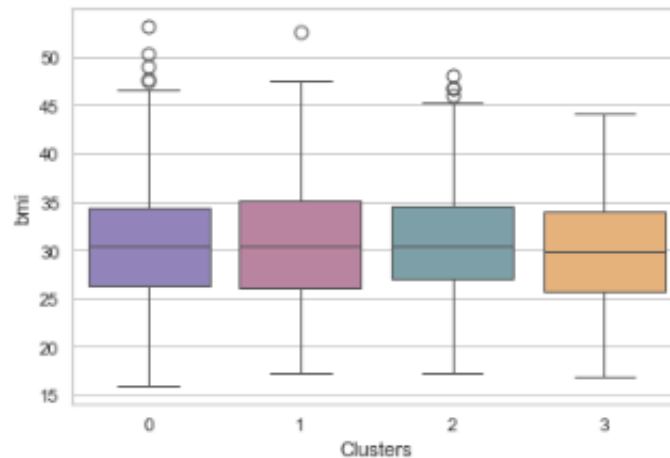
In [16]:

```
1 plt.figure()  
2 pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["charges"], palette=pal)  
3  
4 plt.show()
```



Patient Segmentation

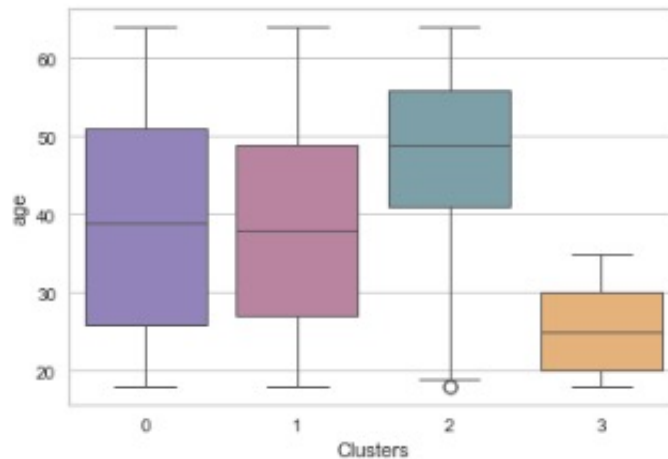
```
In [17]: 1 plt.figure()  
2         plt.figure()  
3         pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["bmi"], palette=pal)  
4         plt.show()
```



Patient Segmentation

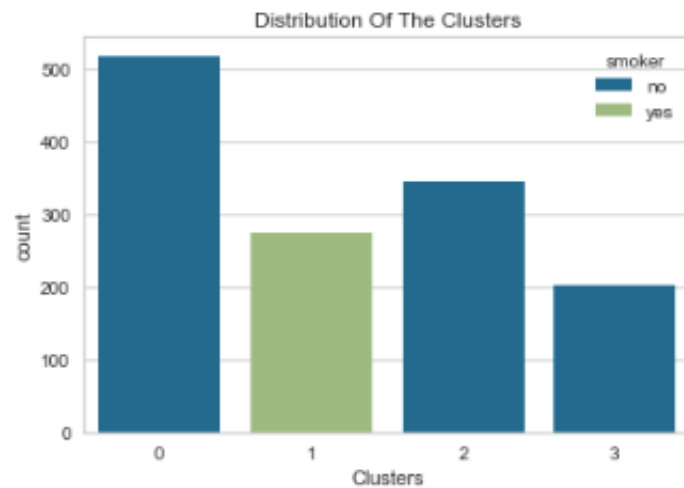
In [18]:

```
1 plt.figure()  
2 pl=sns.boxplot(x=patient_data_original["Clusters"], y=patient_data_original["age"], palette=pal)  
3  
4 plt.show()
```



Patient Segmentation

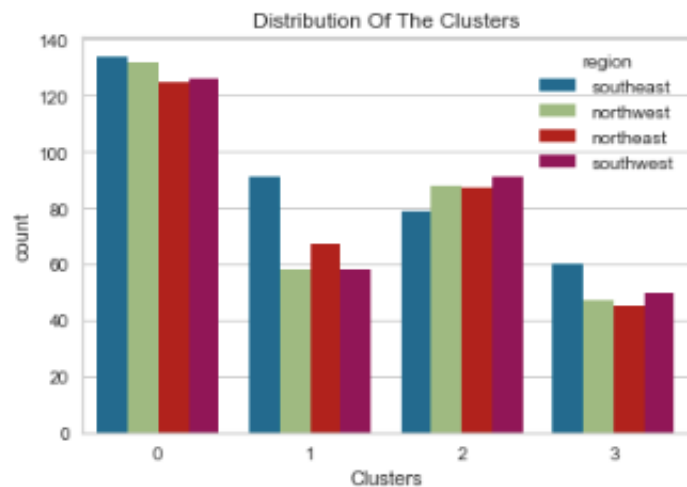
```
In [19]: 1  
2 pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["smoker"])  
3 pl.set_title("Distribution Of The Clusters")  
4 plt.show()
```



Patient Segmentation

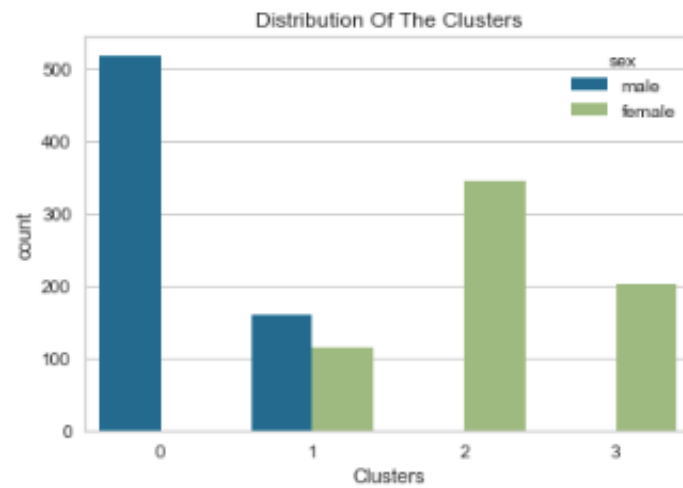
In [20]:

```
1 pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["region"])
2 pl.set_title("Distribution Of The Clusters")
3
4 plt.show()
```



Patient Segmentation

```
In [21]: 1  
2 pl = sns.countplot(x=patient_data_original["Clusters"], hue=patient_data_original["sex"])  
3 pl.set_title("Distribution Of The Clusters")  
4 plt.show()
```



Patient Segmentation- Clustering

Understanding the clusters

Cluster	Characteristics
0	median charges around \$5,000, median age around 40, non_smokers, male
1	median charges around \$35,000, median age around 40, smokers
2	median charges around \$10,000, median age around 50, non_smokers, female
3	median charges around \$2,500, median age around 25, non_smokers, female

References

- <https://www.kaggle.com/datasets/imakash3011/customer-personality-analysis>